2026 하반기 시험 대비

리눅스마스터 2급 학습 자료 합본

11개 문서 · 마크다운 노트를 한 권으로 묶은 PDF

목차

리눅스마스터 2급 — 시험 개요 · 일정 · 합격 전략 2

01. 리눅스 일반 5

02. 리눅스 설치 10

03. 기본 명령어 15

04. 파일시스템과 권한 관리 25

05. 쉘(Shell)과 프로세스 관리 33

06. 에디터와 소프트웨어 설치 45

07. X 윈도와 시스템 관리 53

08. 네트워크 설정과 인터넷 활용 64

09. 명령어 치트시트 (인쇄용) 74

10. 빈출 암기노트 — 시험 전날용 83

쪽 번호는 표지를 제외한 본문 기준이며 각 페이지 하단의 번호와 같습니다. PDF 뷰어의 북마크(개요)로도 이동할 수 있습니다.

리눅스마스터 2급 — 시험 개요 · 일정 · 합격 전략

1. 기본 정보

항목	내용
시행기관	한국정보통신진흥협회(KAIT) — https://www.ihd.or.kr
자격 종류	국가공인 민간자격
응시자격	**제한 없음** (누구나)
시험 횟수	연 4회
학점 인정	5학점
검정 구분	1차 시험 + 2차 시험 (둘 다 합격해야 취득)

2. 시험 구성 — 가장 많이 헷갈리는 부분

구분	1차 시험	2차 시험
방식	**온라인 CBT**	**오프라인 필기(PBT)**
문항 수	50문항	**80문항**
문제 유형	객관식 4지선다	**객관식 4지선다**
시험 시간	60분	100분 (14:00~15:40)
배점	100점	100점
합격 기준	60점 이상 (30문항 이상)	60점 이상 + **과목당 40% 미만 과락**
결과 확인	응시 직후 즉시	지정 발표일

> 🔴 **핵심: 2급의 2차 시험은 실습(실기)이 아니라 객관식 필기입니다.** 터미널에서 직접 작업하는 실습 시험은 리눅스마스터 **1급** 2차입니다. "2급도 실기 실습이 있다"는 잘못된 정보에 시간 낭비하지 마세요.

3. 출제 과목 및 범위

1차 시험 (50문항)

과목	주요 내용
리눅스 일반	리눅스 개요, 역사, 라이선스, 배포판, 디렉터리 구조
리눅스 운영 및 관리	설치, 기본 명령어, 파일시스템, 권한, 프로세스, Shell, 에디터, 패키지
리눅스 활용	X 윈도, 인터넷 활용, 네트워크 설정, 응용 분야

2차 시험 (80문항) — 과목 비중이 중요

과목	문항 수	비중	과락선
1과목: 리눅스 운영 및 관리	48문항	60%	19.2점 미만 과락
2과목: 리눅스 활용	32문항	40%	12.8점 미만 과락

> 1과목 48문항 중 **터미널 명령어 문제가 8할 이상**입니다. 명령어와 옵션 암기가 곧 점수입니다. 2과목은 네트워크·응용 분야여서 범위가 넓고 체감 난이도가 높습니다. **과락 방지를 위해 2과목을 절대 버리지 마세요.**

4. 2026년 시험 일정 (하반기 = 2604회)

회차	1차 접수	1차 시험	2차 접수	2차 시험	최종 발표
2603	07.27~08.05	07.28~08.06	07.28~08.07	09.12	10.09
2604 ★	**10.26~11.04**	**10.27~11.05**	**10.27~11.06**	**12.12**	**12.31**

- 1차는 접수 기간 중 원하는 날짜/시간을 선택해 온라인으로 응시합니다.
- 2026년 2회차부터 1차가 "정해진 시간에 시작하는 온라인 시험" 방식으로 변경되었습니다.
- **2차 접수가 1차 시험과 거의 동시에 진행됩니다(10.27~11.06).** 1차 합격 후 반드시 기간 내 2차를 접수하세요. 놓치면 다음 회차까지 기다려야 합니다.
- 일정은 접수 전 https://www.ihd.or.kr/guidecert1.do 에서 최종 확인.

5. 합격 전략

학습 순서 (권장)

```
01_리눅스 일반          ← 가볍게, 암기 위주 (2일)
02_리눅스 설치          ← 파티션/부트로더 (2일)
03_기본 명령어          ← 여기가 핵심, 시간 많이 투자 (5일)
04_파일시스템과 권한관리 ← 퍼미션 계산 필수 (3일)
05_쉘과 프로세스 관리    ← 3일
06_에디터와 소프트웨어 설치 ← vi + rpm/yum/apt (3일)
07_X윈도와 시스템 관리   ← 2일
08_네트워크 설정과 인터넷 활용 ← 2과목 과락 방지 핵심 (4일)
09_명령어 치트시트       ← 매일 반복
10_빈출 암기노트        ← 시험 전날 1회독
```

실천 수칙

1. **명령어는 눈으로 읽지 말고 직접 타이핑하세요.** WSL2 또는 VirtualBox + Rocky Linux를 깔고 한 번씩 실행해보면 옵션이 머리에 박힙니다.

   ```
   wsl --install -d Ubuntu
   ```

2. **기출 회독이 왕도입니다.** 문제은행 경향이 강해 최근 2~3년 기출에서 유사 문제가 많이 나옵니다. https://www.comcbt.com/xe/r2 에서 무료로 회차별 풀이 가능.
3. **옵션 문자를 구분해서 외우세요.** `tar -zxvf` vs `-jxvf`, `ps -ef` vs `ps aux`, `chmod 755` vs `chmod u+x` 같은 혼동 포인트가 정답을 갈라놓습니다.
4. **2차는 과락을 먼저 막으세요.** 1과목 만점을 노리기보다 2과목에서 13점 이상을 확실히 확보하는 편이 합격에 가깝습니다.
5. **합격률이 20~30%대로 떨어지는 회차도 있습니다.** 벼락치기가 통하는 시험이라는 후기도 많지만, 그건 명령어 암기량이 이미 있는 경우입니다. 2주는 확보하세요.

6. 파일 목록

파일	내용
01_리눅스 일반.md	리눅스 개요, 역사, 라이선스, 배포판
02_리눅스 설치.md	파티션, 파일시스템 종류, 부트로더
03_기본 명령어.md	파일·디렉터리·검색·압축 명령어
04_파일시스템과 권한관리.md	디렉터리 구조, 퍼미션, 링크, 마운트, 쿼터
05_쉘과 프로세스 관리.md	Shell 종류·변수, 프로세스, 작업 스케줄링
06_에디터와 소프트웨어 설치.md	vi/emacs/nano, rpm·yum·dnf·apt, 소스 컴파일
07_X윈도와 시스템 관리.md	X윈도, 계정관리, 로그, 백업, 프린터
08_네트워크 설정과 인터넷 활용.md	네트워크 설정, 서버 서비스, 응용 분야
09_명령어 치트시트.md	인쇄용 명령어·옵션 총정리
10_빈출 암기노트.md	시험 전날용 최종 정리

7. 출처

- 리눅스마스터 자격안내 · 출제기준 (공식)
- 리눅스마스터 시험일정 (공식)
- 2026년 리눅스마스터 2급 시험일정
- 리눅스마스터 - 나무위키
- 리눅스마스터 2급 기출문제 CBT

01. 리눅스 일반

> 암기 위주 과목. 배점은 크지 않지만 **틀리면 아까운 문제**들이라 확실히 잡고 갑니다.

---

1. 리눅스의 탄생과 역사

연도	사건
1969	**UNIX** 탄생 — AT&T 벨 연구소, 켄 톰슨(Ken Thompson)
1972	데니스 리치(Dennis Ritchie)가 **C 언어**로 UNIX 재작성
1983	**리처드 스톨만(Richard Stallman)** — GNU 프로젝트 시작
1985	FSF(Free Software Foundation) 설립
1987	**앤드루 타넨바움** — MINIX 발표 (교육용 UNIX 계열)
1989	**GPL** 발표
1991	**리누스 토발즈(Linus Torvalds)** — 리눅스 커널 0.01 발표 (핀란드 헬싱키 대학)
1992	리눅스 커널이 GPL 라이선스 채택
1994	리눅스 커널 1.0 발표
1996	커널 2.0, 마스코트 **턱스(Tux) 펭귄** 등장
2011	커널 3.0

출제 포인트

- **리눅스 창시자 = 리누스 토발즈 (1991년)**
- **GNU 프로젝트 창시자 = 리처드 스톨만 (1983년)**
- **UNIX 개발자 = 켄 톰슨, 데니스 리치**
- **MINIX 개발자 = 앤드루 타넨바움**
- 리눅스 = **GNU 소프트웨어 + 리눅스 커널** → 정식 명칭 "GNU/Linux"

---

2. 리눅스의 특징

특징	설명
다중 사용자(Multi-user)	여러 사용자가 동시에 시스템 사용
다중 작업(Multi-tasking)	여러 프로세스 동시 실행
다중 처리(Multi-processing)	여러 CPU 동시 활용 (SMP 지원)
다중 플랫폼	x86, ARM, PowerPC 등 다양한 아키텍처
계층적 파일시스템	루트(/)를 최상위로 하는 트리 구조
강력한 네트워킹	TCP/IP 기본 내장
우수한 이식성(Portability)	C로 작성되어 이식 용이
뛰어난 안정성·보안성	사용자·그룹 권한 기반
오픈 소스	소스 공개, 무료 사용·수정·배포

특징	설명
가상 콘솔	하나의 모니터에서 여러 개의 가상 화면

단점

- 공식적인 기술 지원 부재(커뮤니티 의존)
- 상용 소프트웨어 상대적 부족
- 하드웨어 드라이버 지원 지연 가능
- 사용자 학습 곡선

---

3. 라이선스 (매우 중요 — 매 회차 출제)

라이선스	특징	파생물 소스 공개	대표 예
GPL (GNU General Public License)	**Copyleft**. 2차 저작물도 반드시 GPL로 공개	**필수**	리눅스 커널, GCC
LGPL (Lesser GPL)	라이브러리용. 링크만 하면 공개 의무 없음	링크 시 불필요	glibc
BSD	매우 자유로움. 저작권 표시만 요구	불필요	FreeBSD, OpenSSH
Apache	BSD 유사 + 특허권 관련 조항	불필요	Apache HTTP Server
MIT / X11	가장 단순·자유	불필요	X 윈도
MPL (Mozilla Public License)	파일 단위 Copyleft	수정 파일만	Firefox

핵심 구분

- **Copyleft 있음 = GPL** → 소스 공개 의무. "GPL 코드를 쓴 상용 제품도 소스를 공개해야 하는가?" → **예**
- **Copyleft 없음 = BSD, MIT, Apache** → 상용 제품에 넣고 소스 비공개 가능

FSF의 자유 소프트웨어 4대 자유

1. 목적에 상관없이 실행할 자유
2. 소스를 연구하고 변경할 자유
3. 복제하고 재배포할 자유
4. 개선된 버전을 배포할 자유

오픈 소스 관련 용어

- **FSF**: Free Software Foundation (리처드 스톨만)
- **OSI**: Open Source Initiative — 오픈소스 정의(OSD) 관리
- **Copyleft**: 저작권(Copyright)의 반대 개념, 자유를 강제

---

4. 리눅스 커널 버전 체계

구 방식 (2.6 이전)

`주버전.부버전.패치버전` → 예 `2.4.18`

- **부버전이 짝수 = 안정(Stable) 버전** (2.0, 2.2, 2.4)
- **부버전이 홀수 = 개발(Development) 버전** (2.1, 2.3, 2.5)

신 방식 (2.6 이후)

짝수/홀수 구분을 사용하지 않고 `주.부.패치[-추가]` 형태로 관리 예: `3.10.0-1160.el7.x86_64`

커널 확인 명령

```
uname -r          # 커널 버전만
uname -a          # 전체 시스템 정보
cat /proc/version # 커널 + 컴파일러 정보
```

출제 포인트

"2.4.18은 안정 버전인가?" → **예** (부버전 4 = 짝수) "2.5.10은?" → 개발 버전

---

5. 배포판(Distribution)

배포판 = **리눅스 커널 + GNU 유틸리티 + 패키지 관리자 + 설치 프로그램**

계열별 분류 (출제 빈도 높음)

계열	패키지 형식	패키지 관리자	대표 배포판
Red Hat 계열	`.rpm`	`rpm`, `yum`, `dnf`	RHEL, **CentOS**, **Rocky Linux**, AlmaLinux, Fedora
Debian 계열	`.deb`	`dpkg`, `apt`, `apt-get`	Debian, **Ubuntu**, Linux Mint, Kali
Slackware 계열	`.tgz`	`pkgtool`	Slackware, openSUSE(zypper/rpm)
기타	-	`pacman` / `emerge`	Arch Linux / Gentoo

> ⚠️ **함정 주의**: openSUSE는 Slackware에서 파생되었지만 **rpm 패키지**를 사용합니다.

주요 배포판 특징

배포판	특징
Slackware	최초의 상용 배포판 계열, 가장 UNIX에 가까움
Debian	커뮤니티 주도, 안정성 최우선, 방대한 패키지
Red Hat / RHEL	상용 기업용, 유료 기술지원
Fedora	RHEL의 테스트베드, 최신 기술 선도
CentOS	RHEL 소스 기반 무료 클론 (CentOS Stream으로 전환)
Rocky / AlmaLinux	CentOS 정책 변경 후 등장한 RHEL 호환 무료 배포판
Ubuntu	Debian 기반, 데스크톱 친화, LTS 5년 지원
Kali Linux	모의해킹·보안 특화
Android	리눅스 커널 기반 모바일 OS

> 리눅스마스터 2급 실습 환경으로는 **Rocky Linux** 계열이 자주 언급됩니다. 명령어 예제도 RHEL 계열(`yum` / `dnf`, `systemctl`) 기준으로 외우는 것이 유리합니다.

---

6. 리눅스의 활용 분야

분야	내용
서버	웹(Apache/Nginx), DB(MySQL/PostgreSQL), 메일, DNS, 파일, 프록시
임베디드	라우터, 셋톱박스, 스마트 TV, IoT, 라즈베리파이
모바일	Android, Tizen
클라우드/가상화	KVM, Xen, Docker, Kubernetes, OpenStack
HPC(고성능 컴퓨팅)	슈퍼컴퓨터, 클러스터, **베어울프(Beowulf) 클러스터**
데스크톱	Ubuntu, Fedora Workstation

클러스터링 3종 (출제 포인트)

종류	목적	예
고계산용 클러스터(HPC)	계산 성능 향상	베어울프 클러스터
부하분산 클러스터(LVS)	트래픽 분산	Linux Virtual Server
고가용성 클러스터(HA)	무중단 서비스	이중화, 페일오버

가상화 방식

방식	설명	예
전가상화(Full Virtualization)	게스트 OS 수정 불필요, 하드웨어 완전 에뮬레이션	KVM, VMware
반가상화(Para Virtualization)	게스트 OS 수정 필요, 성능 우수	Xen
컨테이너	커널 공유, 프로세스 수준 격리, 가볍고 빠름	Docker, LXC

---

7. 셸(Shell)과 커널의 관계

> 사용자 → 셸(Shell) → 커널(Kernel) → 하드웨어

- **커널**: OS의 핵심. 메모리·프로세스·파일시스템·장치 관리
- **셸**: 사용자 명령을 해석해 커널에 전달하는 **명령어 해석기**
- **유틸리티**: 실제 작업을 수행하는 프로그램들

---

8. 마무리 체크 문제

1. 리눅스 커널 최초 개발자와 연도? → **리누스 토발즈, 1991년**
2. GNU 프로젝트 창시자? → **리처드 스톨만**
3. 2차 저작물의 소스 공개를 강제하는 라이선스? → **GPL**
4. 상용 제품에 포함해도 소스 공개 의무가 없는 라이선스 3개? → BSD, MIT, Apache
5. 커널 2.4.18은 안정 버전인가 개발 버전인가? → **안정(짝수)**
6. `.rpm`을 쓰는 계열과 대표 배포판? → Red Hat 계열, RHEL/CentOS/Rocky/Fedora
7. `.deb`를 쓰는 계열? → Debian 계열, Debian/Ubuntu/Mint
8. 커널 버전을 확인하는 명령? → `uname -r`

9. 게스트 OS 수정이 필요한 가상화 방식? → **반가상화(Xen)**
10. 계산 성능 향상을 목적으로 하는 클러스터? → **고계산용(HPC) / 베어울프**

02. 리눅스 설치

1. 설치 전 준비

하드웨어 정보 확인 항목

CPU 아키텍처(x86/x86_64/ARM), 메모리 용량, 디스크 용량·인터페이스, 그래픽 카드, NIC, 사운드 카드

디스크 장치명 규칙 (출제 포인트)

인터페이스	장치명	예
IDE (PATA)	`/dev/hd[a-d]`	1st Master=hda, 1st Slave=hdb, 2nd Master=hdc, 2nd Slave=hdd
SCSI / SATA / USB / SAS	`/dev/sd[a-z]`	첫 디스크=sda, 두 번째=sdb
NVMe SSD	`/dev/nvme0n1`	파티션은 `nvme0n1p1`
가상 디스크(KVM)	`/dev/vda`	
플로피	`/dev/fd0`	
CD-ROM	`/dev/cdrom`, `/dev/sr0`	

파티션 번호 규칙 (MBR)

번호	종류
1 ~ 4	**주 파티션(Primary) 또는 확장 파티션(Extended)**
5 이상	**논리 파티션(Logical)** — 확장 파티션 안에 생성

예: `/dev/sda1` = 첫 SATA 디스크의 첫 주 파티션 `/dev/sda5` = 첫 논리 파티션

MBR vs GPT

구분	MBR	GPT
최대 디스크 용량	**2TB**	사실상 무제한(9.4ZB)
파티션 개수	주 4개 (확장 사용 시 논리 확장)	128개 이상
부팅 방식	BIOS	**UEFI**
백업	없음	헤더·테이블 이중 저장

2. 파티션 분할

필수 파티션

파티션	설명	권장 크기
`/` (루트)	**반드시 필요**. 최상위 디렉터리	최소 10GB 이상
swap	가상 메모리. 파일시스템 아님	물리 메모리의 1~2배 (일반적으로 RAM ≤ 2GB면 2배, 그 이상은 1~1.5배)

선택 파티션

파티션	분리 이유
`/boot`	커널·부트로더 저장. 예전 BIOS 제약 대응, 200MB~1GB
`/boot/efi`	UEFI 부팅 시 필요(FAT32)
`/home`	사용자 데이터 분리 → 재설치 시 데이터 보존
`/var`	로그·메일·스풀 급증 대비 (서버에서 중요)
`/usr`	응용 프로그램
`/tmp`	임시 파일 폭주 방지

파티션 분할의 장점

- 특정 영역이 꽉 차도 시스템 전체가 멈추지 않음
- 백업·복구 단위 분리
- 마운트 옵션(읽기전용, noexec 등)을 영역별로 다르게 적용
- 파일시스템 손상 시 피해 범위 국소화

LVM (Logical Volume Manager)

> 물리 볼륨(PV) → 볼륨 그룹(VG) → 논리 볼륨(LV)

- 파티션 크기를 **서비스 중단 없이 확장** 가능
- 여러 디스크를 묶어 하나의 큰 볼륨으로 사용
- 주요 명령: `pvcreate`, `vgcreate`, `lvcreate`, `lvextend`, `vgdisplay`

---

3. 파일시스템 종류 (출제 빈도 높음)

파일시스템	특징
ext2	저널링 **없음**
ext3	ext2 + **저널링(Journaling)** 추가, 최대 파일 2TB
ext4	ext3 개선. 최대 볼륨 1EB, 최대 파일 16TB, extent 방식, 지연 할당
XFS	SGI 개발. 대용량 파일·고성능에 강함. **RHEL 7 이후 기본**
Btrfs	스냅샷, 서브볼륨, CoW 지원
ReiserFS	작은 파일 처리에 강함
JFS	IBM 개발
swap	스왑 전용
iso9660	CD/DVD
vfat / FAT32	윈도우 호환
NTFS	윈도우. 리눅스는 ntfs-3g로 지원
NFS	네트워크 파일시스템 (UNIX/Linux 간 공유)
CIFS / SMB	윈도우 공유 접속 (Samba)
procfs (/proc)	커널·프로세스 정보 제공하는 **가상 파일시스템**

파일시스템	특징
sysfs (/sys)	장치·드라이버 정보 가상 파일시스템
tmpfs	메모리 기반 임시 파일시스템

저널링(Journaling)이란?

파일시스템 변경 내용을 먼저 저널 영역에 기록한 뒤 실제 반영. **비정상 종료 시 빠른 복구**가 가능. → **ext2는 저널링 없음 / ext3·ext4·XFS는 저널링 있음** (시험 단골)

---

4. 부트 매니저 (부트로더)

부트로더	특징
LILO (Linux Loader)	구형. 설정 변경 후 `lilo` 명령 재실행 필요. 설정파일 `/etc/lilo.conf`
GRUB (Legacy, 0.9x)	설정파일 `/boot/grub/grub.conf` (또는 menu.lst). 대화형 셸 지원
GRUB2	현재 표준. 설정파일 `/boot/grub2/grub.cfg` — **직접 수정하지 않음**

GRUB2 설정 절차 (중요)

```
vi /etc/default/grub          # 기본 설정 수정
# 또는 /etc/grub.d/ 스크립트 수정
grub2-mkconfig -o /boot/grub2/grub.cfg   # 설정 파일 생성(RHEL 계열)
update-grub                              # (Debian 계열)
```

GRUB Legacy 설정 항목

```
default=0          # 기본 부팅 항목 (0부터 시작)
timeout=5          # 대기 시간(초)
title CentOS       # 메뉴 표시 이름
  root (hd0,0)     # /boot 위치 — hd0=첫 디스크, 0=첫 파티션
  kernel /vmlinuz-... ro root=/dev/sda2
  initrd /initramfs-...
```

> ⚠️ GRUB의 디스크·파티션 번호는 **0부터 시작**합니다. `(hd0,0)` = 첫 디스크의 첫 파티션.

루트 패스워드 분실 시 복구 (단골 문제)

GRUB 메뉴에서 커널 파라미터에 `single`, `1`, 또는 `init=/bin/bash`를 추가해 단일 사용자 모드로 진입 → `passwd root`

---

5. 부팅 과정 (순서 암기)

```
1. BIOS/UEFI — POST 수행, 부팅 장치 선택
2. MBR/GPT — 부트로더(GRUB) 적재
3. GRUB — 커널(vmlinuz) + initramfs 로드
4. 커널 — 하드웨어 초기화, 루트 파일시스템 마운트
5. init / systemd — PID 1 프로세스 실행
6. 런레벨/타깃에 맞는 서비스 시작
7. 로그인 프롬프트(getty / display manager)
```

런레벨 (SysV init) — 반드시 암기

런레벨	의미
0	시스템 종료(halt)
1	단일 사용자 모드(single user) — 복구용
2	다중 사용자 모드, **NFS 미지원**
3	다중 사용자 모드, **텍스트 콘솔(CLI)**
4	미사용(사용자 정의)
5	다중 사용자 모드 + **X 윈도(GUI)**
6	재부팅(reboot)

관련 파일: `/etc/inittab` (기본 런레벨 지정) 관련 명령: `runlevel`, `init 3`, `telinit 5`, `who -r`

systemd 타깃 (런레벨 대응) — 최신 출제

런레벨	systemd 타깃
0	`poweroff.target`
1	`rescue.target`
3	`multi-user.target`
5	`graphical.target`
6	`reboot.target`

```
systemctl get-default                      # 현재 기본 타깃 확인
systemctl set-default multi-user.target    # 기본 타깃 변경
systemctl isolate graphical.target         # 즉시 타깃 전환
```

init vs systemd

구분	SysV init	systemd
PID 1	`init`	`systemd`
실행 방식	스크립트 순차 실행	**병렬 실행** → 부팅 빠름
설정 단위	런레벨 + `/etc/rc.d/` 스크립트	**유닛(Unit)**
서비스 제어	`service httpd start`	`systemctl start httpd`
로그	`/var/log/messages`	`journalctl`

systemd 유닛 종류

`.service` (서비스), `.target` (런레벨 대체), `.socket`, `.mount`, `.timer` (cron 대체), `.device`, `.swap`

---

6. 시스템 종료·재부팅 명령

```
shutdown -h now            # 즉시 종료
shutdown -h +10            # 10분 후 종료
shutdown -r now            # 즉시 재부팅
shutdown -c                # 예약된 종료 취소
shutdown -k "메시지"        # 실제 종료 없이 경고만
halt                       # 종료
poweroff                   # 전원 차단
reboot                     # 재부팅
init 0 / init 6            # 런레벨로 종료/재부팅
systemctl poweroff / reboot # systemd 방식
sync                       # 버퍼 내용을 디스크에 기록 (종료 전 권장)
```

옵션	의미
`-h`	halt (종료)
`-r`	reboot (재시작)
`-c`	cancel (예약 취소)
`-k`	실제 종료 없이 경고 메시지만
`-f`	다음 부팅 시 fsck 생략
`-F`	다음 부팅 시 fsck 강제 수행

---

7. 마무리 체크 문제

1. 첫 번째 SATA 디스크의 세 번째 논리 파티션 장치명? → `/dev/sda7` (5,6,7 순서)
2. MBR의 최대 디스크 인식 용량? → **2TB**
3. 반드시 필요한 파티션 2개? → `/` 와 **swap**
4. ext2와 ext3의 결정적 차이? → **저널링 유무**
5. RHEL 7 이후 기본 파일시스템? → **XFS**
6. GRUB2 설정 파일 생성 명령(RHEL)? → `grub2-mkconfig -o /boot/grub2/grub.cfg`
7. GUI를 포함한 다중 사용자 런레벨? → **5**
8. 런레벨 5에 해당하는 systemd 타깃? → `graphical.target`
9. `(hd0,1)`이 의미하는 것? → 첫 번째 디스크의 두 번째 파티션
10. 10분 후 시스템을 종료하는 명령? → `shutdown -h +10`

03. 기본 명령어

> **이 단원이 시험의 절반입니다.** 2차 1과목 48문항 중 대부분이 명령어 문제입니다. 명령어 자체보다 **옵션 문자**를 정확히 구분하는 것이 핵심입니다.

---

1. 디렉터리 이동·확인

```
pwd             # 현재 디렉터리 절대경로 출력
cd /etc         # 절대경로 이동
cd ..           # 상위 디렉터리
cd ~            # 홈 디렉터리 (cd 만 입력해도 동일)
cd ~user1       # user1의 홈 디렉터리
cd -            # 직전 디렉터리로 복귀
```

기호	의미
`.`	현재 디렉터리
`..`	상위 디렉터리
`~`	현재 사용자 홈 디렉터리
`~user`	특정 사용자 홈 디렉터리
`-`	이전 작업 디렉터리
`/`	루트 디렉터리

---

2. ls — 목록 출력

```
ls -l           # 상세 정보(long)
ls -a           # 숨김 파일 포함(all, . 으로 시작하는 파일)
ls -A           # 숨김 포함하되 . 과 .. 제외
ls -d           # 디렉터리 자체 정보 (내용 아님)
ls -h           # 사람이 읽기 쉬운 단위(K, M, G)
ls -i           # inode 번호 출력
ls -R           # 하위 디렉터리까지 재귀적
ls -t           # 수정 시간 순 정렬
ls -S           # 파일 크기 순 정렬
ls -r           # 역순 정렬
ls -F           # 파일 유형 표시 (/ 디렉터리, * 실행, @ 링크)
ls -lrt         # 자주 쓰는 조합: 시간 역순 상세
```

`ls -l` 출력 해석 (시험 단골)

```
-rw-r--r--.  1 root root  1234 Sep 23 10:00 test.txt
 | └──┬──┘ └┬┘ └┬┘    |   |     |      |         |         |
 |    |     |   |     |   |     |      |         |         └ 파일명
 |    |     |   |     |   |     |      |         └ 최종 수정 시간
 |    |     |   |     |   |     |      └ 파일 크기(바이트)
 |    |     |   |     |   |     └ 그룹
 |    |     |   |     |   └ 소유자
 |    |     |   |     └ 하드링크 수
 |    |     |   └ other 권한
 |    |     └ group 권한
 |    └ user(소유자) 권한
 └ 파일 종류
```

첫 문자(파일 종류)

기호	의미
`-`	일반 파일
`d`	디렉터리
`l`	**심볼릭 링크**
`b`	블록 장치 (디스크)
`c`	문자 장치 (터미널, 프린터)
`p`	파이프(FIFO)
`s`	소켓

---

3. 파일·디렉터리 생성·삭제·복사·이동

```
touch file.txt          # 빈 파일 생성 / 기존 파일의 시간 갱신
touch -t 202612120900 f # 시간 지정
mkdir dir1              # 디렉터리 생성
mkdir -p a/b/c          # 중간 경로까지 한 번에 생성 (parents)
mkdir -m 755 dir        # 권한 지정 생성
rmdir dir1              # 빈 디렉터리만 삭제
rmdir -p a/b/c          # 상위까지 삭제

rm file                 # 파일 삭제
rm -r dir               # 디렉터리 재귀 삭제 (recursive)
rm -f file              # 강제 삭제, 확인 없음 (force)
rm -i file              # 삭제 전 확인 (interactive)
rm -rf dir              # 재귀 강제 삭제 ← 가장 위험

cp a b                  # 복사
cp -r dir1 dir2         # 디렉터리 복사 (필수 옵션)
cp -i                   # 덮어쓰기 전 확인
cp -p                   # 속성(소유자 · 권한 · 시간) 보존 (preserve)
cp -a                   # 아카이브 = -dpR (원본 그대로)
cp -u                   # 원본이 더 최신일 때만 복사 (update)
cp -f                   # 강제
cp -v                   # 진행 상황 출력 (verbose)

mv a b                  # 이동 또는 이름 변경
```

```
mv -i                   # 덮어쓰기 확인
mv -f                   # 강제
mv -u                   # 최신일 때만
mv -b                   # 덮어쓸 파일 백업
```

출제 포인트

- **디렉터리 복사는 `-r` 필수**, `cp dir1 dir2` 만 하면 오류
- `mv` 는 같은 파일시스템 내에서는 이름만 바뀌므로 inode가 유지됨
- `rmdir` 은 **빈 디렉터리만** 삭제 가능 → 내용이 있으면 `rm -r`

---

4. 파일 내용 보기

```
cat file                # 전체 출력
cat -n file             # 행 번호 표시
cat -b file             # 빈 줄 제외하고 번호 표시
cat f1 f2 > f3          # 연결(concatenate)
tac file                # 역순 출력 (cat의 반대)

more file               # 페이지 단위 (앞으로만)
less file               # 페이지 단위 (앞/뒤 모두, more보다 강력)

head file               # 앞 10줄 (기본)
head -n 5 file          # 앞 5줄
head -5 file            # 동일
tail file               # 뒤 10줄
tail -n 20 file         # 뒤 20줄
tail -f /var/log/messages  # 실시간 추적 (follow) ★로그 확인 단골

nl file                 # 행 번호 붙여 출력
wc file                 # 행/단어/바이트 수
wc -l file              # 행(line) 수만
wc -w file              # 단어(word) 수
wc -c file              # 바이트(character) 수
od file                 # 8진수 등으로 덤프
file test.txt           # 파일 종류 판별
strings bin             # 바이너리에서 문자열 추출
```

출제 포인트

- `tail -f` = 로그 실시간 모니터링 (가장 많이 나옴)
- `wc -l` = 행 수 (`-l` 이 line)
- `less` vs `more`: less는 **뒤로 스크롤 가능**

---

5. 검색 — find (매우 중요)

```
find /home -name "*.txt"          # 이름으로 검색
find / -iname "abc*"              # 대소문자 무시
find /tmp -type f                 # 파일만 (d=디렉터리, l=링크)
find / -user user1                # 소유자로 검색
find / -group admin               # 그룹으로 검색
find / -perm 644                  # 권한이 정확히 644
find / -perm -4000                # SetUID가 설정된 파일 ★보안 점검 단골
find / -size +10M                 # 10MB 초과
find / -size -100k                # 100KB 미만
find / -empty                     # 빈 파일/디렉터리
find / -mtime -7                  # 7일 이내 수정된 파일
find / -mtime +30                 # 30일 이전 수정
find / -atime -1                  # 1일 이내 접근
find / -newer ref.txt             # ref.txt보다 최신
find / -maxdepth 2 -name "*.conf"  # 탐색 깊이 제한
find /tmp -name "*.tmp" -exec rm -f {} \;   # 찾아서 실행 ★
find /tmp -name "*.log" -delete    # 찾아서 삭제
find . -name "a*" -o -name "b*"    # OR 조건
find . ! -name "*.txt"            # 부정(NOT)
```

시간 옵션 3종 구분 (자주 출제)

옵션	기준
`-atime`	**A**ccess — 마지막 접근(읽기) 시각
`-mtime`	**M**odify — 마지막 내용 수정 시각
`-ctime`	**C**hange — 마지막 속성(inode) 변경 시각

`+n` = n일보다 이전, `-n` = n일 이내, `n` = 정확히 n일 전

다른 검색 명령

```
which ls          # PATH 내 실행 파일 경로
whereis ls        # 실행파일 + 소스 + man 페이지 경로
locate passwd     # DB 기반 빠른 검색 (updatedb로 DB 갱신)
updatedb          # locate용 DB 갱신
type cd           # 명령어 유형(내장/외부/별칭) 확인
```

명령	특징
`find`	실시간 디스크 탐색 → 느리지만 정확
`locate`	**DB 검색** → 매우 빠르지만 DB가 오래되면 부정확
`which`	PATH 환경변수 내에서만 검색
`whereis`	바이너리·소스·매뉴얼 위치

6. 문자열 검색 — grep

```
grep "root" /etc/passwd
grep -i "ROOT" file        # 대소문자 무시 (ignore case)
grep -v "root" file        # 일치하지 않는 행 (invert) ★
grep -n "root" file        # 행 번호 표시
grep -c "root" file        # 일치하는 행 개수만 (count)
grep -l "root" *           # 일치하는 파일명만 출력
grep -r "root" /etc        # 하위 디렉터리 재귀 검색
grep -w "root" file        # 단어 단위로 정확히 일치
grep -E "a|b" file         # 확장 정규표현식 (= egrep)
grep -F "lit" file         # 고정 문자열 (= fgrep)
grep -A 3 "err" file       # 일치 행 뒤 3줄 함께
grep -B 3 "err" file       # 일치 행 앞 3줄 함께
```

정규표현식 기초

기호	의미
`^`	행의 시작
`$`	행의 끝
`.`	임의의 한 문자
`*`	앞 문자 0회 이상 반복
`[abc]`	a 또는 b 또는 c
`[^abc]`	a, b, c 가 아닌 문자
`[a-z]`	범위
`\`	이스케이프
`+`	(확장) 1회 이상 반복
`?`	(확장) 0 또는 1회
`|`	(확장) OR

예: `grep "^root" /etc/passwd` → root로 시작하는 행 `grep "bash$" /etc/passwd` → bash로 끝나는 행

---

7. 압축과 아카이브 (시험 단골 ★★★)

압축 명령

명령	확장자	압축 해제	특징
`compress`	`.Z`	`uncompress`	구형
`gzip`	`.gz`	`gunzip`, `gzip -d`	가장 일반적
`bzip2`	`.bz2`	`bunzip2`, `bzip2 -d`	gzip보다 압축률 높음
`xz`	`.xz`	`unxz`, `xz -d`	압축률 최고
`zip`	`.zip`	`unzip`	윈도우 호환, **여러 파일 묶기 가능**

핵심 차이: `gzip`, `bzip2`, `xz`는 **파일 하나만** 압축(묶기 불가). 여러 파일을 묶으려면 `tar`가 필요합니다. `zip`은 묶기+압축을 동시에 합니다.

```
gzip file          # file → file.gz (원본 삭제)
gzip -d file.gz    # 압축 해제
gzip -l file.gz    # 압축 정보 확인 (list)
gzip -r dir        # 디렉터리 내 파일 각각 압축
gzip -9 file       # 최대 압축률 (1=빠름, 9=최대)
zcat file.gz       # 압축 해제 없이 내용 보기
```

tar — 아카이브 (반드시 암기)

```
tar cvf backup.tar dir/        # 묶기 (create)
tar xvf backup.tar             # 풀기 (extract)
tar tvf backup.tar             # 목록 보기 (list) ★
tar rvf backup.tar newfile     # 기존 아카이브에 추가 (append)
tar uvf backup.tar file        # 갱신된 것만 추가 (update)

tar zcvf backup.tar.gz dir/    # gzip 압축과 함께 묶기 ★★
tar zxvf backup.tar.gz         # gzip 압축 풀기 ★★
tar jcvf backup.tar.bz2 dir/   # bzip2 압축
tar jxvf backup.tar.bz2        # bzip2 해제
tar Jcvf backup.tar.xz dir/    # xz 압축 (대문자 J)
tar Jxvf backup.tar.xz         # xz 해제

tar xvf b.tar -C /opt          # 지정 디렉터리에 풀기
tar cvf b.tar --exclude=*.log dir/   # 특정 파일 제외
```

옵션	의미
`c`	create — 새 아카이브 생성
`x`	extract — 풀기
`t`	list — 내용 목록 보기
`r`	append — 뒤에 추가
`u`	update — 갱신된 파일만 추가
`v`	verbose — 과정 출력
`f`	file — 아카이브 파일명 지정 (**거의 항상 필요**)
`z`	**gzip** 연동 (`.tar.gz`, `.tgz`)
`j`	**bzip2** 연동 (`.tar.bz2`)
`J`	**xz** 연동 (`.tar.xz`)
`p`	권한 보존
`C`	대상 디렉터리 지정

⚠️ **최다 출제 함정**: `z`(gzip)와 `j`(bzip2)와 `J`(xz) 구분. 그리고 `t`가 "목록 보기"라는 점.

---

8. 링크 — ln

```
ln origin hardlink       # 하드 링크
ln -s origin symlink     # 심볼릭(소프트) 링크 ★
```

구분	하드 링크	심볼릭 링크
inode	**원본과 동일**	별도 inode
원본 삭제 시	**데이터 유지** (링크로 접근 가능)	**깨짐**(dangling)
다른 파일시스템	**불가**	가능
디렉터리 대상	**불가**	가능
`ls -l` 표시	`-` (일반 파일)	`l` + `-> 원본`
링크 수 카운트	증가	증가하지 않음
윈도우 비유	-	바로가기

> 출제 포인트: **하드 링크는 디렉터리와 다른 파일시스템에 만들 수 없다.**

---

9. 리다이렉션과 파이프

```
command > file        # 표준출력 → 파일 (덮어쓰기)
command >> file       # 표준출력 → 파일 (이어쓰기)
command < file        # 파일 → 표준입력
command 2> file       # 표준에러 → 파일
command > file 2>&1   # 표준출력 + 표준에러 → 같은 파일 ★
command &> file       # 위와 동일(bash)
command 2> /dev/null  # 에러 메시지 버리기
command1 | command2   # 파이프 — 앞 출력을 뒤 입력으로
```

표준 스트림 번호 (암기)

번호	이름	기본 대상
0	표준 입력 (stdin)	키보드
1	표준 출력 (stdout)	모니터
2	표준 에러 (stderr)	모니터

tee — 화면 출력과 파일 저장 동시에

```
ls -l | tee out.txt       # 화면에도 보이고 파일에도 저장
ls -l | tee -a out.txt    # 이어쓰기
```

---

10. 텍스트 처리 필터

```
sort file             # 오름차순 정렬
sort -r file          # 내림차순 (reverse)
sort -n file          # 숫자 기준 정렬 (numeric)
sort -k 3 file        # 3번째 필드 기준
```

```
sort -u file          # 중복 제거 정렬
sort -t: -k3 -n /etc/passwd   # 구분자 : , 3번째 필드 숫자 정렬

uniq file             # 연속된 중복 행 제거 (정렬 후 사용)
uniq -c file          # 중복 횟수와 함께
uniq -d file          # 중복된 행만
uniq -u file          # 중복 없는 행만

cut -d: -f1 /etc/passwd     # : 구분, 1번 필드 추출 ★
cut -d: -f1,3 /etc/passwd   # 1,3번 필드
cut -c1-5 file              # 1~5번째 문자

tr 'a-z' 'A-Z' < file       # 소문자 → 대문자 변환
tr -d '\r' < file           # 특정 문자 삭제
tr -s ' ' < file            # 연속 문자 하나로 압축

paste f1 f2           # 두 파일을 행 단위로 붙이기
join f1 f2            # 공통 필드로 결합
split -l 100 file     # 100행 단위로 분할
diff f1 f2            # 두 파일 차이 비교
cmp f1 f2             # 바이너리 비교
comm f1 f2            # 정렬된 두 파일 공통/차이

sed 's/old/new/' file       # 첫 번째 매칭만 치환
sed 's/old/new/g' file      # 전체 치환 (global) ★
sed -i 's/old/new/g' file   # 파일 직접 수정 (in-place)
sed -n '5,10p' file         # 5~10행만 출력
sed '3d' file               # 3행 삭제

awk '{print $1}' file           # 1번째 필드 출력
awk -F: '{print $1, $3}' passwd  # 구분자 지정
awk '{print NR, $0}' file       # 행 번호와 전체 행
awk '$3 > 500 {print $1}' file   # 조건부 출력
awk 'END {print NR}' file       # 총 행 수
```

awk 내장 변수

변수	의미
`$0`	전체 행
`$1`, `$2` …	n번째 필드
`NR`	현재 행 번호 (Number of Records)
`NF`	현재 행의 필드 개수 (Number of Fields)
`FS`	입력 필드 구분자
`OFS`	출력 필드 구분자

---

11. 기타 유용한 명령

```
date                  # 현재 날짜 · 시간
date +%Y-%m-%d        # 형식 지정
cal                   # 달력
cal 12 2026           # 특정 월 달력
uptime                # 가동 시간 + 부하 평균
who                   # 현재 로그인 사용자
```

```
w                       # 로그인 사용자 + 작업 내용
whoami                  # 현재 사용자명
id                      # UID, GID, 그룹 정보
last                    # 최근 로그인 기록
users                   # 로그인 사용자명 목록
hostname                # 호스트명
uname -a                # 시스템 전체 정보
free -h                 # 메모리 사용량
df -h                   # 디스크 여유 공간 ★
du -sh dir              # 디렉터리 사용량 (summary, human) ★
echo "hello"            # 출력
printf "%s\n" "hi"      # 형식 출력
clear                   # 화면 지우기
history                 # 명령 이력
alias ll='ls -l'        # 별칭 설정
unalias ll              # 별칭 해제
man ls                  # 매뉴얼
man 5 passwd            # 섹션 지정 매뉴얼
info ls                 # info 문서
ls --help               # 간단한 도움말
```

`df` vs `du` (혼동 주의)

- `df` = **d**isk **f**ree → **파일시스템 단위** 여유 공간
- `du` = **d**isk **u**sage → **디렉터리/파일 단위** 사용량

man 페이지 섹션 번호

섹션	내용
1	사용자 명령어
2	시스템 콜
3	라이브러리 함수
4	장치 파일 (`/dev`)
5	**설정 파일 형식** (`/etc/passwd` 등)
6	게임
7	기타
8	**관리자 명령어**
9	커널 루틴

---

12. 마무리 체크 문제

1. 디렉터리를 복사하려면 필수 옵션? → `-r`
2. `tar` 로 `.tar.bz2` 를 푸는 명령? → `tar jxvf`
3. `tar` 로 아카이브 **내용만 확인**하는 옵션? → `t`
4. 로그 파일을 실시간으로 보는 명령? → `tail -f`
5. 일치하지 않는 행만 출력하는 grep 옵션? → `-v`
6. SetUID가 설정된 파일을 찾는 명령? → `find / -perm -4000`
7. 하드 링크를 디렉터리에 만들 수 있는가? → **아니오**

8. 원본을 삭제해도 데이터가 살아있는 링크? → **하드 링크**
9. 표준 에러의 파일 디스크립터 번호? → **2**
10. 파일시스템 여유 공간을 보는 명령? → `df -h`
11. `/etc/passwd`에서 사용자명만 뽑는 명령? → `cut -d: -f1 /etc/passwd`
12. 파일 내 문자열을 전부 치환? → `sed -i 's/old/new/g' file`
13. `-atime`, `-mtime`, `-ctime` 각각의 기준? → 접근 / 내용 수정 / 속성 변경
14. `locate`가 `find`보다 빠른 이유? → **미리 만들어둔 DB를 검색**하기 때문
15. 설정 파일 형식을 설명하는 man 섹션 번호? → **5**

04. 파일시스템과 권한 관리

1. 리눅스 디렉터리 계층 구조 (FHS) — 암기 필수

```
/
├──── bin      기본 실행 명령어 (모든 사용자, ls · cp · mv)
├──── boot     커널(vmlinuz), initramfs, 부트로더(GRUB)
├──── dev      장치 파일 (sda, tty, null, zero)
├──── etc      ★ 시스템 설정 파일 (passwd, fstab, hosts)
├──── home     일반 사용자 홈 디렉터리
├──── lib      공유 라이브러리, 커널 모듈
├──── media    이동식 장치 자동 마운트 지점 (USB, CD)
├──── mnt      임시 수동 마운트 지점
├──── opt      추가 응용 프로그램 (third-party)
├──── proc     ★ 가상 파일시스템 — 커널 · 프로세스 정보 (메모리 상 존재)
├──── root     root 계정의 홈 디렉터리 (/ 와 다름!)
├──── run      런타임 데이터 (PID 파일 등)
├──── sbin     ★ 관리자용 실행 명령어 (fdisk, ifconfig, shutdown)
├──── srv      서비스 데이터
├──── sys      가상 파일시스템 — 장치 · 드라이버 정보
├──── tmp      임시 파일 (누구나 쓰기 가능, Sticky Bit 설정됨)
├──── usr      사용자 공용 프로그램 (usr/bin, usr/sbin, usr/local)
└──── var      ★ 가변 데이터 — 로그(log), 메일(mail), 스풀(spool), 캐시
```

시험에 나오는 구분

질문	답
시스템 설정 파일이 모인 곳?	`/etc`
로그 파일이 저장되는 곳?	`/var/log`
관리자 전용 명령어 위치?	`/sbin` , `/usr/sbin`
커널이 저장되는 곳?	`/boot`
장치 파일 위치?	`/dev`
메모리에만 존재하는 가상 파일시스템?	`/proc` , `/sys`
root의 홈 디렉터리?	`/root`
누구나 쓸 수 있는 임시 디렉터리?	`/tmp`

`/proc` 주요 파일

파일	내용
`/proc/cpuinfo`	CPU 정보
`/proc/meminfo`	메모리 정보
`/proc/version`	커널 버전
`/proc/partitions`	파티션 정보
`/proc/mounts`	마운트 정보
`/proc/interrupts`	IRQ 정보
`/proc/[PID]/`	개별 프로세스 정보

---

2. 파일 권한(Permission) — 최다 출제 ★★★

권한 구조

```
rwx  rwx  rwx
 |    |    └──── other (기타 사용자)
 |    └─────────── group (그룹)
 └──────────────── user  (소유자)
```

권한	기호	8진수	파일에서의 의미	디렉터리에서의 의미
읽기	`r`	**4**	내용 읽기	목록 보기(`ls`)
쓰기	`w`	**2**	내용 수정	파일 생성/삭제
실행	`x`	**1**	실행	**진입(`cd`)**

8진수 변환 연습 (반드시 즉답 가능해야 함)

8진수	기호	의미
777	`rwxrwxrwx`	모두 모든 권한
755	`rwxr-xr-x`	소유자 전체, 나머지 읽기+실행 (**실행파일·디렉터리 표준**)
644	`rw-r--r--`	소유자 읽기쓰기, 나머지 읽기 (**일반 파일 표준**)
700	`rwx------`	소유자만
600	`rw-------`	소유자만 읽기쓰기 (SSH 키)
666	`rw-rw-rw-`	모두 읽기쓰기
400	`r--------`	소유자 읽기만
111	`--x--x--x`	모두 실행만

변환법: `rwx` → 4+2+1 = 7, `r-x` → 4+0+1 = 5, `r--` → 4

chmod — 권한 변경

```
# 8진수 방식
chmod 755 file
chmod 644 *.txt
chmod -R 755 dir/        # 하위까지 재귀 적용

# 기호 방식
chmod u+x file           # 소유자에게 실행 권한 추가
chmod g-w file           # 그룹 쓰기 권한 제거
chmod o=r file           # other를 읽기만으로 설정
chmod a+x file           # 모두(all)에게 실행 추가
chmod ug+rw,o-rwx file   # 복합 지정
chmod u+s file           # SetUID 설정
chmod g+s dir            # SetGID 설정
chmod o+t dir            # Sticky Bit 설정
chmod --reference=a b    # a의 권한을 b에 복사
```

대상	기호
user (소유자)	`u`
group (그룹)	`g`
other (기타)	`o`
all (전체)	`a`

연산자	의미
`+`	권한 추가
`-`	권한 제거
`=`	권한 지정(나머지 제거)

chown / chgrp — 소유권 변경

```
chown user1 file           # 소유자 변경
chown user1:group1 file    # 소유자 + 그룹 변경 ★
chown user1.group1 file    # 동일 (점 표기)
chown :group1 file         # 그룹만 변경
chown -R user1 dir/        # 재귀 적용
chgrp group1 file          # 그룹만 변경
```

> `chown` 은 **root만** 실행 가능(일반 사용자는 자기 파일의 소유자를 남에게 넘길 수 없음).

---

3. umask — 기본 권한 마스크

```
umask          # 현재 값 확인 (예: 0022)
umask -S       # 기호로 확인
umask 0077     # 값 변경
```

계산 방법 (시험 단골)

대상	기준값	계산
파일	**666**	666 - umask
디렉터리	**777**	777 - umask

> 파일에 기준값 666을 쓰는 이유: 새로 만든 파일에는 **실행 권한을 기본으로 주지 않기 때문**.

예제

umask	파일 권한	디렉터리 권한
022	666-022 = **644**	777-022 = **755**
002	**664**	**775**
077	**600**	**700**
000	**666**	**777**
027	**640**	**750**

- root 기본 umask: **022**
- 일반 사용자 기본 umask: **002** (RHEL 계열, UPG 정책)
- 설정 위치: `/etc/profile`, `/etc/bashrc`, `~/.bashrc`

---

4. 특수 권한 — SetUID / SetGID / Sticky Bit ★★★

특수 권한	8진수	표시 위치	의미
SetUID	**4**	user 실행 자리 → `s`	실행 중 **파일 소유자 권한**으로 동작
SetGID	**2**	group 실행 자리 → `s`	실행 중 **파일 그룹 권한**으로 동작 / 디렉터리에 설정하면 생성 파일이 그 그룹을 상속
Sticky Bit	**1**	other 실행 자리 → `t`	디렉터리 내 파일을 **소유자만 삭제** 가능

설정 예

```
chmod 4755 file      # SetUID  → -rwsr-xr-x
chmod 2755 dir       # SetGID  → drwxr-sr-x
chmod 1777 dir       # Sticky  → drwxrwxrwt
chmod 6755 file      # SetUID + SetGID
chmod u+s file       # 기호 방식 SetUID
chmod o+t dir        # 기호 방식 Sticky
```

대표 예시 (시험 단골)

대상	권한	특수 권한
`/usr/bin/passwd`	`-rwsr-xr-x` (4755)	**SetUID** — 일반 사용자가 `/etc/shadow`를 수정할 수 있게 함
`/tmp`	`drwxrwxrwt` (1777)	**Sticky Bit** — 아무나 쓸 수 있지만 남의 파일은 못 지움
`/var/tmp`	`drwxrwxrwt`	Sticky Bit

대문자 S / T 의 의미 (함정 문제)

- **소문자** `s` / `t` = 특수 권한 + 실행 권한(x) 둘 다 있음
- **대문자** `S` / `T` = 특수 권한만 있고 **실행 권한(x)이 없음** → 실제로는 동작하지 않음

예: `chmod 4655 file` → `-rwSr-xr-x` (실행 권한 없어 SetUID 무효)

SetUID 파일 검색 (보안 점검)

```
find / -perm -4000 -type f         # SetUID
find / -perm -2000 -type f         # SetGID
find / -perm -1000 -type d         # Sticky Bit
find / -perm -6000                 # SetUID + SetGID
```

---

5. inode와 파일 속성

inode에 저장되는 정보

파일 종류, 권한, 소유자 UID/GID, 크기, 링크 수, 타임스탬프(atime/mtime/ctime), **데이터 블록 위치**

inode에 저장되지 않는 것 (출제 포인트)

파일 이름 — 파일명은 디렉터리 엔트리에 저장됩니다.

```
ls -i file            # inode 번호 확인
df -i                 # 파일시스템별 inode 사용량
stat file             # 상세 속성(3가지 시간 포함)
```

확장 속성 — chattr / lsattr

```
chattr +i file        # immutable — 삭제 · 수정 불가 (root도) ★
chattr -i file        # 해제
chattr +a file        # append only — 추가만 가능(로그 보호)
lsattr file           # 속성 확인
```

> `chattr +i` 가 걸린 파일은 root도 삭제할 수 없습니다. 시험 단골.

---

6. 마운트와 /etc/fstab

```
mount                           # 현재 마운트 목록
mount /dev/sdb1 /mnt/data       # 수동 마운트
mount -t ext4 /dev/sdb1 /data   # 파일시스템 지정
mount -o ro /dev/sdb1 /data     # 읽기 전용
mount -o remount,rw /           # 재마운트
mount -a                        # /etc/fstab 전체 마운트
umount /mnt/data                # 마운트 해제
umount -l /mnt/data             # lazy umount
```

> 사용 중인 디렉터리는 언마운트되지 않습니다(`device is busy`). `fuser -cu /mnt` 또는 `lsof /mnt` 로 사용 중인 프로세스를 확인하세요.

/etc/fstab 필드 구성 (6개 — 순서 암기)

```
/dev/sda2   /home   ext4   defaults   1   2
   (1)       (2)    (3)     (4)     (5) (6)
```

필드	이름	설명
1	장치명	`/dev/sda2`, `UUID=...`, `LABEL=...`
2	마운트 포인트	`/home` (swap은 `swap` 또는 `none`)
3	파일시스템 종류	ext4, xfs, swap, nfs
4	마운트 옵션	defaults, ro, rw, noexec, nosuid, noauto, usrquota
5	**dump**	백업 여부 (0=안함, 1=함)
6	**fsck 순서**	0=검사안함, **1=루트(/)**, 2=나머지

주요 마운트 옵션

옵션	의미
`defaults`	rw, suid, dev, exec, auto, nouser, async
`ro` / `rw`	읽기전용 / 읽기쓰기
`noexec`	실행 파일 실행 금지
`nosuid`	SetUID 무시
`noauto`	부팅 시 자동 마운트 안 함
`user`	일반 사용자도 마운트 가능
`usrquota` / `grpquota`	쿼터 사용

---

7. 파일시스템 생성·점검

```
fdisk -l                  # 파티션 목록
fdisk /dev/sdb            # 파티션 편집(대화형)
  n(new) p(primary) w(write) d(delete) t(type) p(print) q(quit)
parted /dev/sdb           # GPT 지원 파티션 도구
gdisk /dev/sdb            # GPT 전용

mkfs -t ext4 /dev/sdb1    # 파일시스템 생성
mkfs.ext4 /dev/sdb1       # 동일
mkfs.xfs /dev/sdb1        # XFS 생성
mke2fs -j /dev/sdb1       # ext3 생성(-j = 저널)

mkswap /dev/sdb2          # 스왑 영역 생성
swapon /dev/sdb2          # 스왑 활성화
swapoff /dev/sdb2         # 스왑 비활성화
swapon -s                 # 스왑 상태 확인 (= cat /proc/swaps)

fsck /dev/sdb1            # 파일시스템 검사 · 복구 ★
fsck -y /dev/sdb1         # 모든 질문에 yes
e2fsck -f /dev/sdb1       # ext 계열 강제 검사
xfs_repair /dev/sdb1      # XFS 복구
tune2fs -l /dev/sdb1      # ext 파일시스템 정보 조회
tune2fs -c 30 /dev/sdb1   # 최대 마운트 횟수 설정
```

```
dumpe2fs /dev/sdb1          # 상세 정보
blkid                       # UUID · 파일시스템 타입 확인 ★
lsblk                       # 블록 장치 트리
```

⚠️ **fsck 는 반드시 언마운트 상태에서 실행**해야 합니다. 마운트된 상태에서 실행하면 파일시스템이 손상될 수 있습니다.

---

8. 디스크 쿼터(Quota)

사용자·그룹별 디스크 사용량을 제한하는 기능.

```
# 1. /etc/fstab에 usrquota,grpquota 옵션 추가 후 재마운트
# 2. 쿼터 DB 파일 생성
quotacheck -mcug /home
# 3. 쿼터 활성화
quotaon /home
quotaoff /home
# 4. 사용자별 설정
edquota -u user1            # 편집기로 한도 설정
edquota -g group1           # 그룹 쿼터
edquota -t                  # grace period(유예기간) 설정
edquota -p user1 user2      # user1 설정을 user2에 복사
setquota -u user1 100M 200M 0 0 /home
# 5. 확인
quota -u user1              # 사용자 쿼터 확인
repquota /home              # 전체 보고서 ★
repquota -a                 # 모든 파일시스템
```

용어

용어	의미
soft limit	초과 가능하지만 경고 발생, grace period 동안만 허용
hard limit	**절대 초과 불가**
grace period	soft limit 초과 후 허용되는 기간(기본 7일)
blocks	용량 제한
inodes	파일 개수 제한

---

9. 마무리 체크 문제

1. `rw-r--r--` 를 8진수로? → **644**
2. `755` 를 기호로? → `rwxr-xr-x`
3. umask가 022일 때 새 파일·디렉터리 권한? → **644 / 755**
4. umask가 027일 때 새 디렉터리 권한? → **750**
5. `/usr/bin/passwd` 에 설정된 특수 권한? → **SetUID (4755)**
6. `/tmp` 에 설정된 특수 권한과 이유? → **Sticky Bit**, 남의 파일 삭제 방지
7. `drwxrwxrwT` 의 의미? → Sticky Bit 설정 but other 실행 권한 없음
8. inode에 저장되지 않는 정보? → **파일 이름**
9. root도 수정·삭제할 수 없게 만드는 명령? → `chattr +i`

10. `/etc/fstab`의 5번째, 6번째 필드? → **dump 여부 / fsck 순서**
11. `/etc/fstab`에서 루트 파일시스템의 fsck 순서 값? → **1**
12. 파일시스템 UUID를 확인하는 명령? → `blkid`
13. `fsck` 실행 시 주의사항? → **반드시 언마운트 후 실행**
14. 절대 초과할 수 없는 쿼터 한도의 이름? → **hard limit**
15. 쿼터 사용 현황 보고서 명령? → `repquota`

05. 쉘(Shell)과 프로세스 관리

1. 쉘의 종류와 특징

쉘	개발자	프롬프트	특징
sh (Bourne Shell)	스티븐 본(Stephen Bourne)	`$`	최초의 표준 쉘
csh (C Shell)	빌 조이(Bill Joy)	`%`	C 언어 문법, **히스토리·별칭 기능 최초 도입**
ksh (Korn Shell)	데이비드 콘(David Korn)	`$`	sh 호환 + csh 기능
bash (Bourne Again Shell)	브라이언 폭스(Brian Fox)	`$`	**리눅스 기본 쉘**, sh 호환 + 확장
tcsh	-	`%`	csh 개선판
zsh	폴 팔스타드	`%`	강력한 자동완성
dash	-	`$`	경량, Ubuntu의 `/bin/sh`

출제 포인트

- **리눅스 기본 쉘 = bash, 개발자 = 브라이언 폭스**
- **히스토리·별칭 기능을 처음 도입한 쉘 = csh (빌 조이)**
- root의 기본 쉘도 `/bin/bash`

쉘 관련 명령

```
echo $SHELL            # 현재 로그인 쉘
cat /etc/shells        # 사용 가능한 쉘 목록 ★
chsh                   # 로그인 쉘 변경(대화형)
chsh -s /bin/csh user1 # 특정 쉘로 변경
chsh -l                # 사용 가능 쉘 목록
usermod -s /bin/bash u # 쉘 변경(관리자)
exit / logout          # 쉘 종료
```

> 로그인 쉘 정보는 `/etc/passwd`의 **7번째 필드**에 저장됩니다. 로그인을 막으려면 쉘을 `/sbin/nologin` 또는 `/bin/false`로 지정합니다.

2. 쉘 변수와 환경 변수

```
VAR="hello"            # 지역(쉘) 변수 선언 — 공백 없이!
echo $VAR              # 값 출력
export VAR             # 환경 변수로 승격 ★
export VAR="hi"        # 선언과 동시에 환경변수
unset VAR              # 변수 해제
set                    # 모든 쉘 변수 + 함수 목록
env                    # 환경 변수만 목록 ★
printenv               # 환경 변수 출력
printenv PATH          # 특정 환경변수
declare -x VAR         # export와 동일
readonly VAR           # 읽기 전용 변수
```

구분	지역 변수	환경 변수
자식 프로세스 상속	**안 됨**	**됨**
확인 명령	`set`	`env` , `printenv`
선언	`VAR=값`	`export VAR=값`

주요 환경 변수 (암기)

변수	의미
`PATH`	실행 파일 검색 경로 (`:` 로 구분)
`HOME`	홈 디렉터리
`USER` / `LOGNAME`	사용자명
`SHELL`	로그인 쉘
`PWD` / `OLDPWD`	현재 / 이전 디렉터리
`PS1`	**기본 프롬프트 문자열**
`PS2`	2차 프롬프트 (`>`)
`TERM`	터미널 종류
`LANG`	언어·로케일 (`ko_KR.UTF-8`)
`HISTSIZE`	히스토리 저장 개수
`HISTFILE`	히스토리 파일 (`~/.bash_history`)
`HOSTNAME`	호스트명
`MAIL`	메일 저장 경로
`DISPLAY`	X 윈도 디스플레이
`UID` / `EUID`	사용자 ID
`TMOUT`	자동 로그아웃 시간(초)

PS1 프롬프트 특수 문자 (출제 포인트)

기호	의미
`\u`	사용자명
`\h`	호스트명(짧게) / `\H` 전체
`\w`	현재 경로 전체 / `\W` 마지막 디렉터리만
`\d`	날짜 / `\t` 24시간 시간 / `\T` 12시간
`\$`	root면 `#` , 일반 사용자면 `$`
`\n`	줄바꿈
`\!`	히스토리 번호

예: `PS1='[\u@\h \W]\$ '` → `[user1@localhost ~]$`

특수 쉘 변수

변수	의미
`$0`	실행된 스크립트/쉘 이름
`$1` ~ `$9`	위치 인자
`$#`	인자의 개수
`$*` / `$@`	전체 인자
`$?`	**직전 명령의 종료 상태** (0=성공) ★
`$$`	현재 쉘의 PID
`$!`	마지막 백그라운드 프로세스 PID

---

3. 쉘 환경 설정 파일 (읽는 순서 중요)

전역 설정 (모든 사용자)

파일	역할
`/etc/profile`	로그인 시 실행. 환경변수·PATH·umask 설정
`/etc/profile.d/*.sh`	profile이 불러오는 개별 스크립트
`/etc/bashrc` (또는 `/etc/bash.bashrc`)	bash 실행 시마다. 별칭·함수

개인 설정 (사용자별, 홈 디렉터리)

파일	역할
`~/.bash_profile`	**로그인 시 1회** 실행. 환경변수
`~/.bash_login`	bash_profile 없을 때
`~/.profile`	위 둘 없을 때
`~/.bashrc`	**bash 실행할 때마다**. 별칭·함수 ★
`~/.bash_logout`	로그아웃 시 실행
`~/.bash_history`	명령 이력 저장

읽는 순서 (로그인 쉘)

> `/etc/profile` → `/etc/profile.d/*.sh` → `~/.bash_profile` → `~/.bashrc` → `/etc/bashrc`

출제 포인트

- **환경 변수는 `~/.bash_profile`, 별칭(alias)은 `~/.bashrc`**
- 설정 파일 수정 후 즉시 적용: `source ~/.bashrc` 또는 `. ~/.bashrc`

---

4. 명령 이력(History)

```
history                 # 전체 이력
history 10              # 최근 10개
history -c              # 이력 삭제(clear)
history -w              # 메모리 이력을 파일에 기록
!!                      # 직전 명령 재실행 ★
!52                     # 52번 명령 재실행
!ls                     # ls로 시작하는 최근 명령
!$                      # 직전 명령의 마지막 인자
!^                      # 직전 명령의 첫 인자
^old^new                # 직전 명령에서 old를 new로 바꿔 실행
Ctrl+R                  # 역방향 검색
```

관련 환경변수: `HISTSIZE` (메모리 개수), `HISTFILESIZE` (파일 개수), `HISTFILE`, `HISTCONTROL` (ignoredups 등)

---

5. 별칭(alias)과 함수

```
alias                       # 목록
alias ll='ls -alF'          # 설정
alias rm='rm -i'            # 안전장치
unalias ll                  # 해제
\rm file                    # 별칭 무시하고 원래 명령 실행 ★
```

영구 적용: `~/.bashrc` 에 추가

---

6. 쉘 메타문자와 명령 연결

기호	의미
`*`	0개 이상의 모든 문자
`?`	임의의 1문자
`[abc]`	괄호 안 문자 중 하나
`[!abc]` / `[^abc]`	괄호 안 문자 제외
`{a,b}`	중괄호 확장 (`file{1,2}` → file1 file2)
`;`	순차 실행 (결과 무관)
`&&`	앞 명령 **성공** 시에만 뒤 실행
`||`	앞 명령 **실패** 시에만 뒤 실행
`&`	**백그라운드 실행**
`|`	파이프
`` ` ` `` 또는 `$()`	명령 치환
`''`	강한 인용 — 모든 특수문자 무시
`""`	약한 인용 — `$`, `` ` ``, `\` 는 해석됨
`\`	다음 한 문자 이스케이프
`#`	주석

```
echo "오늘은 $(date +%F)"      # 명령 치환
cp file.txt file.txt.bak
mkdir test && cd test          # 성공 시에만 이동
ls nofile || echo "없음"       # 실패 시 메시지
```

---

7. 프로세스 개념

용어	의미
PID	프로세스 ID
PPID	부모 프로세스 ID
UID/EUID	실행 사용자 ID
데몬(Daemon)	백그라운드 상주 서비스 프로세스 (`httpd` , `sshd`)
포그라운드	터미널을 점유하고 실행
백그라운드	터미널과 무관하게 실행 (`&`)
고아 프로세스(Orphan)	부모가 먼저 종료된 프로세스 → init/systemd가 인수
좀비 프로세스(Zombie)	**종료됐지만 부모가 종료 상태를 회수하지 않은** 프로세스. 상태 `Z` , `defunct` 표시

PID 1

- SysV init 시스템: `init`
- systemd 시스템: `systemd`

프로세스 생성 방식 (출제 포인트)

방식	설명
fork	부모 프로세스를 **복제**해 자식 생성 (새 PID 부여)
exec	**현재 프로세스를 새 프로그램으로 덮어씀** (PID 유지)

데몬 실행 방식

방식	설명
standalone	부팅 시 상주하며 직접 요청 처리. 빠름. (`httpd` , `sshd`)
inetd / xinetd (슈퍼 데몬)	요청이 올 때만 해당 데몬 실행. 메모리 절약. 설정: `/etc/xinetd.conf` , `/etc/xinetd.d/`

---

8. 프로세스 확인 — ps / top

```
ps                    # 현재 터미널의 프로세스
ps -e                 # 모든 프로세스 (every)
ps -f                 # 상세 정보 (full)
ps -ef                # ★ 가장 많이 쓰는 조합 (System V 방식)
ps aux                # ★ BSD 방식 (a=모든터미널, u=사용자정보, x=터미널없는것도)
ps -ef | grep httpd   # 특정 프로세스 검색
ps -u user1           # 특정 사용자의 프로세스
ps -l                 # 긴 형식
ps -eo pid,ppid,cmd   # 출력 필드 지정
ps --forest           # 트리 형태
pstree                # 프로세스 계층 트리 ★
pstree -p             # PID 포함
pgrep httpd           # 이름으로 PID 검색
pidof httpd           # 이름으로 PID 검색
```

`ps aux` vs `ps -ef` (혼동 주의)

- `aux` = **BSD 스타일** (하이픈 없음)
- `-ef` = **System V 스타일** (하이픈 있음)
- 둘 다 "모든 프로세스 상세 출력"이라는 목적은 동일

`ps -ef` 출력 읽는 법 (네트워크관리사 2026-3회 41번)

```
UID    PID  PPID  C STIME  TTY      TIME     CMD
icqa  1868     1  0 8월25  ?     00:00:00 /usr/sbin/sshd
icqa  1869  1868  0 8월25  ?     00:00:00 /bin/sh -c /home/icqa/project/venv/bin/python ...
```

열	의미
UID	프로세스를 실행한 사용자
PID	프로세스 ID
PPID	**부모** 프로세스 ID (부모 PID)
C	CPU 사용률(스케줄링용)
STIME	시작 시각
TTY	제어 터미널 (`?` = 터미널 없음, 데몬 등)
TIME	누적 CPU 사용 시간
CMD	실행 명령

> 읽기 요령: **같은 행의 PID와 PPID를 혼동하지 마세요.** 위 예에서 두 번째 행의 PID는 1869이고 PPID는 1868입니다. 프로세스 종료는 **해당 행의 PID**로 `kill PID` 입니다 (PPID가 아님). 기출 41번: '네 번째 프로세스의 PPID는 1869'(PID와 혼동)가 옳지 않은 설명이었습니다. `ps -ef | grep 패턴` 으로 검색하면 `grep` 자신의 행도 함께 나올 수 있습니다.

프로세스 상태 코드 (STAT)

코드	의미
`R`	Running — 실행 중
`S`	Sleeping — 인터럽트 가능 대기

코드	의미
`D`	Uninterruptible sleep — I/O 대기
`T`	Stopped — 중지
`Z`	**Zombie — 좀비**
`<`	높은 우선순위
`N`	낮은 우선순위
`s`	세션 리더
`+`	포그라운드 프로세스 그룹
`l`	멀티스레드

top — 실시간 모니터링

```
top                 # 실행
top -d 3            # 3초 간격 갱신
top -u user1        # 특정 사용자만
top -b -n 1         # 배치 모드(스크립트용)
```

top 내부 대화형 키

키	기능
`h`	도움말
`k`	**프로세스 종료(kill)**
`r`	**우선순위 변경(renice)**
`M`	메모리 사용량 순 정렬
`P`	CPU 사용량 순 정렬
`T`	실행 시간 순 정렬
`u`	특정 사용자 필터
`1`	CPU 코어별 표시
`c`	전체 명령줄 표시
`q`	종료
`space`	즉시 갱신

기타 모니터링

```
uptime              # 부하 평균(load average) 1분/5분/15분
vmstat 1            # 가상 메모리 통계
iostat              # 디스크 I/O
sar                 # 시스템 활동 보고 (sysstat 패키지)
free -h             # 메모리
lsof                # 열린 파일 목록
lsof -i :80         # 80번 포트 사용 프로세스 ★
lsof -u user1       # 사용자별
fuser -v /mnt       # 특정 파일/디렉터리 사용 프로세스
```

---

9. 프로세스 종료 — kill / signal

```
kill PID              # 기본 TERM(15) 시그널
kill -9 PID           # 강제 종료 (KILL)
kill -15 PID          # 정상 종료 요청 (TERM)
kill -l               # 시그널 목록 확인 ★
kill -HUP PID         # 설정 재읽기
killall httpd         # 이름으로 모두 종료
pkill -u user1        # 사용자의 프로세스 종료
pkill -f "pattern"    # 명령줄 패턴으로 종료
```

주요 시그널 (번호 암기 ★★★)

번호	이름	의미
1	**SIGHUP**	터미널 연결 종료 / **데몬 설정 재읽기**
2	**SIGINT**	인터럽트 — **Ctrl + C**
3	SIGQUIT	종료 + 코어 덤프 — Ctrl + \
9	**SIGKILL**	**강제 종료 (무시 불가)**
11	SIGSEGV	세그먼테이션 오류
15	**SIGTERM**	**정상 종료 요청 (kill 기본값)**
18	SIGCONT	중지된 프로세스 계속 실행
19	**SIGSTOP**	프로세스 중지 (무시 불가)
20	SIGTSTP	터미널에서 중지 — **Ctrl + Z**

출제 포인트

- `kill`의 **기본 시그널은 15(SIGTERM)**, `-9`가 아님
- **9(SIGKILL)와 19(SIGSTOP)는 무시하거나 가로챌 수 없음**
- Ctrl+C = 2(INT), Ctrl+Z = 20(TSTP)

---

10. 작업 제어(Job Control)

```
command &             # 백그라운드로 실행
jobs                  # 작업 목록 (작업번호 확인)
jobs -l               # PID 포함
fg                    # 가장 최근 작업을 포그라운드로
fg %1                 # 1번 작업을 포그라운드로 ★
bg %1                 # 1번 작업을 백그라운드에서 계속
Ctrl + Z              # 현재 작업 중지(suspend)
Ctrl + C              # 현재 작업 강제 종료
nohup command &       # 로그아웃 후에도 계속 실행 ★ (출력은 nohup.out)
disown %1             # 쉘 종료와 작업 연결 해제
```

작업 번호 표기

기호	의미
`%n`	n번 작업
`%+` 또는 `%%`	현재 작업
`%-`	이전 작업

`jobs` 출력의 `+`는 현재 작업, `-`는 이전 작업입니다.

---

11. 프로세스 우선순위 — nice / renice

- **NI (nice 값) 범위: -20 ~ 19**
- **값이 작을수록 우선순위 높음** (-20이 최우선)
- 기본값: **0**
- **우선순위를 높이는 것(음수 지정)은 root만 가능**

```
nice -n 10 command        # nice 값 10으로 실행
nice -10 command          # 동일
nice --10 command         # nice 값 -10 (root만)
renice 5 -p 1234          # 실행 중인 PID의 nice 값 변경
renice -n 5 -p 1234       # 동일
renice 5 -u user1         # 사용자의 모든 프로세스
renice -5 -p 1234         # 우선순위 높이기(root만)
ps -l                     # PRI, NI 열로 확인
```

열	의미
PRI	커널이 계산한 실제 우선순위 (낮을수록 높음)
NI	사용자가 지정한 nice 값

---

12. 작업 예약 — cron / at

cron — 반복 예약

```
crontab -e            # 편집 ★
crontab -l            # 목록 확인
crontab -r            # 전체 삭제 (주의!)
crontab -u user1 -l   # 특정 사용자 확인(root)
```

crontab 형식 (필드 5개 — 반드시 암기)

```
분  시  일  월  요일  실행할_명령
*   *   *   *   *
|   |   |   |   └──── 요일 (0~7, 0과 7=일요일, 1=월요일)
|   |   |   └──────── 월   (1~12)
|   |   └──────────── 일   (1~31)
|   └──────────────── 시   (0~23)
└──────────────────── 분   (0~59)
```

기호	의미
`*`	모든 값
`,`	목록 (`1,15`)
`-`	범위 (`1-5`)
`/`	간격 (`*/10` = 10마다)

예제 (시험 단골)

표현	의미
`0 3 * * *`	매일 새벽 3시 0분
`30 2 1 * *`	매월 1일 02:30
`*/10 * * * *`	10분마다
`0 */2 * * *`	2시간마다 정각
`0 9 * * 1`	매주 월요일 09:00
`0 0 * * 0`	매주 일요일 자정
`15 14 1 * *`	매월 1일 14:15
`0 22 * * 1-5`	평일 22:00
`5 4 * * sun`	매주 일요일 04:05

cron 관련 파일

파일/디렉터리	역할
`/etc/crontab`	시스템 crontab (**사용자 필드 추가로 6번째 필드에 실행 계정**)
`/etc/cron.d/`	추가 cron 설정
`/var/spool/cron/`	사용자별 crontab 저장 위치
`/etc/cron.allow`	사용 허용 사용자 목록
`/etc/cron.deny`	사용 금지 사용자 목록
`/etc/cron.hourly` `.daily` `.weekly` `.monthly`	주기별 스크립트 디렉터리
데몬	`crond`

> ⚠️ **`/etc/crontab` 의 필드는 6개**(분 시 일 월 요일 **사용자** 명령), 사용자 crontab은 5개입니다. 함정 문제로 나옵니다.

at — 1회성 예약

```
at 10:00                    # 오늘 10시 예약(대화형, Ctrl+D로 종료)
at 5pm tomorrow
at now + 2 hours
at 10:00 2026-12-12
atq                         # 예약 목록 (= at -l)
atrm 3                      # 3번 작업 삭제 (= at -d 3)
at -f script.sh 10:00       # 파일로 지정
```

관련 파일: `/etc/at.allow` , `/etc/at.deny` , 데몬 `atd` , 스풀 `/var/spool/at`

cron vs at

구분	cron	at
실행	**반복**	**1회성**
데몬	`crond`	`atd`
목록	`crontab -l`	`atq`
삭제	`crontab -r`	`atrm`

---

13. systemd 서비스 관리 (현대 리눅스 필수)

```
systemctl start httpd           # 시작
systemctl stop httpd            # 정지
systemctl restart httpd         # 재시작
systemctl reload httpd          # 설정만 다시 읽기
systemctl status httpd          # 상태 확인 ★
systemctl enable httpd          # 부팅 시 자동 시작 등록 ★
systemctl disable httpd         # 자동 시작 해제
systemctl is-enabled httpd      # 자동시작 여부
systemctl is-active httpd       # 동작 여부
systemctl list-units            # 유닛 목록
systemctl list-unit-files       # 유닛 파일 목록
systemctl mask httpd            # 완전 차단(시작 불가)
systemctl daemon-reload         # 유닛 파일 변경 반영
journalctl -u httpd             # 서비스 로그
journalctl -f                   # 실시간 로그
journalctl -b                   # 현재 부팅 이후 로그
```

구 방식 대응표

SysV	systemd
`service httpd start`	`systemctl start httpd`
`chkconfig httpd on`	`systemctl enable httpd`
`chkconfig --list`	`systemctl list-unit-files`

유닛 파일 위치: `/usr/lib/systemd/system/` (패키지 제공), `/etc/systemd/system/` (관리자 정의, 우선)

---

14. 마무리 체크 문제

1. 리눅스 기본 쉘과 개발자? → **bash / 브라이언 폭스**
2. 히스토리·별칭을 처음 도입한 쉘? → **csh (빌 조이)**
3. 사용 가능한 쉘 목록 파일? → `/etc/shells`
4. 환경변수는 어느 파일에, 별칭은 어느 파일에? → `~/.bash_profile` / `~/.bashrc`
5. 직전 명령의 성공 여부를 담은 변수? → `$?`
6. `kill` 의 기본 시그널 번호와 이름? → **15, SIGTERM**
7. 무시할 수 없는 시그널 2개? → **9(SIGKILL), 19(SIGSTOP)**
8. Ctrl+Z가 보내는 시그널? → **20 (SIGTSTP)**
9. 좀비 프로세스의 STAT 코드? → **Z**

10. nice 값의 범위와 최우선 값? → **-20 ~ 19, -20**
11. 우선순위를 높이려면 누구여야 하는가? → **root**
12. 로그아웃 후에도 실행을 유지하는 명령? → `nohup`
13. `30 4 * * 0` 의 의미? → **매주 일요일 04:30**
14. 10분마다 실행하는 crontab 표현? → `*/10 * * * *`
15. `/etc/crontab`의 필드 개수와 6번째 필드? → **6개, 실행 사용자**
16. 1회성 예약 명령과 목록 확인 명령? → `at` / `atq`
17. 부팅 시 서비스 자동 시작을 등록하는 systemd 명령? → `systemctl enable`
18. `fork`와 `exec`의 차이? → fork는 복제(새 PID), exec는 현재 프로세스 덮어쓰기(PID 유지)

06. 에디터와 소프트웨어 설치

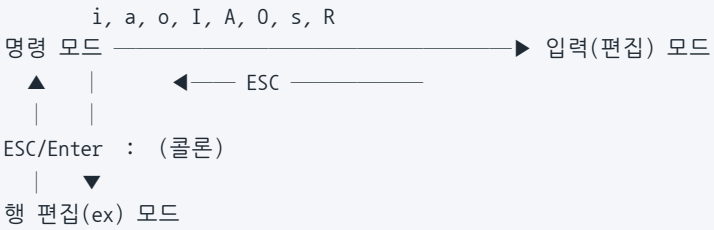
1. 에디터 종류

에디터	개발자	특징
vi	**빌 조이(Bill Joy)**	UNIX 표준 에디터, 모드 기반
vim	**브람 무레나르(Bram Moolenaar)**	vi improved — 구문 강조, 다중 undo
emacs	**리처드 스톨만(Richard Stallman)**	매우 강력, Lisp 확장, 통합 환경
pico	아보일 카사디아스	간단, Pine 메일러 부속
nano	-	pico의 GNU 대체판, 초보자 친화
gedit	-	GNOME 계열 GUI 에디터
kwrite / kate	-	KDE 계열 GUI 에디터
ed / ex	켄 톰슨	라인 단위 에디터

출제 포인트

- **vi = 빌 조이**, **vim = 브람 무레나르**, **emacs = 리처드 스톨만** (이 3개 조합이 단골)
- `nano` 는 `pico` 를 대체하기 위해 만든 GNU 에디터

2. vi 에디터 — 3가지 모드



모드	진입	역할
명령 모드(Command)	기본 / ESC	커서 이동, 삭제, 복사
입력 모드(Insert)	`i` , `a` , `o` 등	실제 텍스트 입력
행 편집 모드(ex/Last line)	`:`	저장, 종료, 치환, 설정

3. vi 입력 모드 진입 키 (구분 중요)

키	동작
`i`	커서 **앞**에 삽입 (insert)
`I`	**행의 맨 앞**에 삽입
`a`	커서 **뒤**에 삽입 (append)
`A`	**행의 맨 끝**에 삽입

키	동작
`o`	커서 **아래**에 새 행 추가 (open)
`O`	커서 **위**에 새 행 추가
`s`	커서 위치 한 글자 삭제 후 입력
`S`	현재 행 전체 삭제 후 입력
`R`	덮어쓰기(Replace) 모드
`cw`	단어 변경(change word)

---

4. vi 커서 이동

키	동작
`h` `j` `k` `l`	좌 / 하 / 상 / 우
`w`	다음 단어의 첫 글자
`b`	이전 단어의 첫 글자
`e`	단어의 끝
`0` (숫자 영)	**행의 처음**
`^`	행의 첫 글자(공백 제외)
`$`	**행의 끝**
`gg`	**문서 맨 처음**
`G`	**문서 맨 끝**
`nG` 또는 `:n`	n번째 행으로 이동
`H` `M` `L`	화면 상단 / 중간 / 하단
`Ctrl+f` / `Ctrl+b`	한 화면 앞으로 / 뒤로
`Ctrl+d` / `Ctrl+u`	반 화면 아래 / 위
`%`	짝이 되는 괄호로 이동

---

5. vi 삭제·복사·붙여넣기

키	동작
`x`	커서 위치 한 글자 삭제
`X`	커서 앞 한 글자 삭제
`nx`	n개 문자 삭제
`dd`	**현재 행 삭제(잘라내기) ★**
`ndd`	n개 행 삭제 (`3dd` = 3행 삭제)
`dw`	단어 삭제
`d$` 또는 `D`	커서부터 행 끝까지 삭제
`d0`	커서부터 행 처음까지 삭제

키	동작
`dG`	커서부터 문서 끝까지 삭제
`yy`	**현재 행 복사(yank) ★**
`nyy`	n개 행 복사
`yw`	단어 복사
`p`	**커서 아래(뒤)에 붙여넣기**
`P`	**커서 위(앞)에 붙여넣기**
`u`	**undo (실행 취소)**
`U`	현재 행의 변경 취소
`Ctrl+r`	redo (다시 실행)
`.`	**직전 명령 반복**
`J`	현재 행과 다음 행 합치기
`~`	대소문자 전환

---

6. vi 검색과 치환

```
/pattern      아래 방향 검색
?pattern      위 방향 검색
n             다음 검색 결과
N             반대 방향 다음 결과
```

치환 (:s 명령 — 시험 단골 ★★)

```
:s/old/new/           현재 행의 첫 번째만 치환
:s/old/new/g          현재 행 전체 치환
:%s/old/new/g         ★ 문서 전체 치환 (% = 전체 행)
:%s/old/new/gc        전체 치환 + 각각 확인(confirm)
:1,10s/old/new/g      1~10행만 치환
:.,$s/old/new/g       현재 행부터 끝까지
:g/pattern/d          pattern이 포함된 행 전체 삭제
```

기호	의미
`%`	문서 전체 행
`.`	현재 행
`$`	마지막 행
`g`	global — 행 내 모든 일치
`c`	confirm — 치환 시 확인
`i`	대소문자 무시

---

7. vi 저장과 종료

명령	동작
`:w`	저장(write)
`:w newfile`	다른 이름으로 저장
`:q`	종료(quit) — 변경사항 있으면 거부
`:wq`	**저장 후 종료 ★**
`:wq!`	강제 저장 후 종료
`ZZ`	`:wq` 와 동일 (명령 모드에서)
`:q!`	**저장하지 않고 강제 종료 ★**
`:x`	변경된 경우만 저장 후 종료
`:e!`	저장 후 마지막 상태로 되돌리기
`:r file`	다른 파일 내용 삽입(read)
`:r !date`	명령 실행 결과 삽입
`:!command`	쉘 명령 실행
`:sh`	임시로 쉘로 나가기 (`exit` 로 복귀)

---

8. vi 환경 설정

```
:set nu          행 번호 표시 (number)
:set nonu        행 번호 제거
:set ai          자동 들여쓰기 (autoindent)
:set ts=4        탭 크기 4 (tabstop)
:set sw=4        들여쓰기 폭 (shiftwidth)
:set ic          대소문자 무시 검색 (ignorecase)
:set hls         검색 결과 강조 (hlsearch)
:set list        비출력 문자 표시
:set showmode    현재 모드 표시
:set ruler       커서 위치 표시
:set all         모든 설정 보기
```

영구 적용: `~/.vimrc` (vim) 또는 `~/.exrc` (vi)

```
# ~/.vimrc 예
set nu
set ai
set ts=4
syntax on
```

vi 복구 (출제 포인트)

비정상 종료 시 스왑 파일(`.filename.swp`)이 생성됩니다.

```
vi -r filename       # 복구 모드로 열기
```

---

9. 패키지 관리 — 개요

계열	저수준(단일 패키지)	고수준(의존성 자동 해결)
Red Hat 계열 (`.rpm`)	`rpm`	`yum` → `dnf`
Debian 계열 (`.deb`)	`dpkg`	`apt-get` , `apt` , `aptitude`
Slackware (`.tgz`)	`installpkg`	`slackpkg`
SUSE (`.rpm`)	`rpm`	`zypper`
Arch	`pacman`	`pacman`

핵심 차이 (시험 단골)

- `rpm` / `dpkg` **는 의존성을 자동 해결하지 못합니다.** 의존 패키지가 없으면 오류.
- `yum` / `dnf` / `apt` **는 저장소(repository)에서 의존성까지 자동 설치합니다.**

---

10. rpm (Red Hat Package Manager)

패키지 파일명 규칙

```
httpd-2.4.6-97.el7.centos.x86_64.rpm
  |       |      |        |          └──── 아키텍처 (x86_64, i686, noarch, src)
  |       |      |        └─────── 배포판(el7 = Enterprise Linux 7)
  |       |      └──────── 릴리스 번호
  |       └─────── 버전
  └──────── 패키지 이름
```

설치·삭제·업그레이드

```
rpm -i pkg.rpm          # 설치(install)
rpm -ivh pkg.rpm        # ★ 설치 + 상세 + 진행바 (가장 많이 씀)
rpm -U pkg.rpm          # 업그레이드 (없으면 새로 설치)
rpm -Uvh pkg.rpm        # ★
rpm -F pkg.rpm          # freshen — 기존에 설치된 것만 갱신
rpm -e pkg              # 삭제(erase) — 패키지명만 씀 (.rpm 아님!)
rpm -e --nodeps pkg     # 의존성 무시 삭제
rpm -ivh --force pkg    # 강제 설치
rpm -ivh --test pkg     # 실제 설치 없이 테스트
```

옵션	의미
`-i`	install
`-U`	Upgrade (없으면 설치)
`-F`	Freshen (있을 때만 갱신)
`-e`	erase (삭제)
`-v`	verbose
`-h`	hash — `#` 진행 표시
`--nodeps`	의존성 무시
`--force`	강제

옵션	의미
`--test`	테스트만

질의(query) — `-q` 계열 ★

```
rpm -qa                # 설치된 전체 패키지 목록 (all)
rpm -qa | grep httpd   # 특정 패키지 설치 확인
rpm -qi httpd          # 패키지 정보 (info)
rpm -ql httpd          # 패키지에 포함된 파일 목록 (list) ★
rpm -qd httpd          # 문서 파일 목록
rpm -qc httpd          # 설정 파일 목록
rpm -qf /etc/httpd/conf/httpd.conf   # ★ 특정 파일이 어느 패키지 소속인지 (file)
rpm -qR httpd          # 의존성 확인 (Requires)
rpm -qp pkg.rpm -i     # 설치 전 rpm 파일 조회 (package)
rpm -qpl pkg.rpm       # 설치 전 파일 목록 확인
rpm -V httpd           # 패키지 검증 (verify) — 변경된 파일 확인
rpm -Va                # 전체 검증
rpm --import KEY       # GPG 키 등록
```

출제 포인트

- **`rpm -qf 파일경로`** = "이 파일은 어느 패키지에 속하는가?" → 매우 자주 출제
- **`rpm -ql 패키지`** = "이 패키지는 어떤 파일을 설치했는가?"
- 삭제할 때는 **패키지명**(예: `httpd`), 설치할 때는 **파일명**(예: `httpd-2.4.rpm`)

---

11. yum / dnf (Red Hat 계열 고수준)

```
yum install httpd          # 설치 (의존성 자동)
yum -y install httpd       # 확인 없이 설치
yum remove httpd           # 삭제 (= erase)
yum update                 # 전체 업데이트
yum update httpd           # 특정 패키지 업데이트
yum check-update           # 업데이트 가능 목록
yum list                   # 전체 패키지 목록
yum list installed         # 설치된 것만
yum list available         # 설치 가능한 것
yum info httpd             # 정보
yum search web             # 이름 · 설명 검색
yum provides */httpd.conf  # ★ 특정 파일을 제공하는 패키지 찾기
yum deplist httpd          # 의존성 목록
yum grouplist              # 패키지 그룹 목록
yum groupinstall "Development Tools"   # 그룹 설치 ★
yum groupremove "..."      # 그룹 삭제
yum clean all              # 캐시 삭제
yum repolist               # 저장소 목록
yum history                # 작업 이력
yum localinstall pkg.rpm   # 로컬 rpm을 의존성 해결하며 설치
```

- 저장소 설정 파일: **`/etc/yum.repos.d/*.repo`**
- 주 설정 파일: `/etc/yum.conf`
- **dnf**는 yum의 차세대 버전으로 명령 체계가 거의 동일합니다(RHEL 8+ 기본).

```
dnf install httpd
dnf group install "Development Tools"
```

---

12. dpkg / apt (Debian 계열)

```
# dpkg — 저수준
dpkg -i pkg.deb            # 설치
dpkg -r pkg                # 삭제 (설정 파일 유지)
dpkg -P pkg                # 완전 삭제 (purge)
dpkg -l                    # 설치된 패키지 목록
dpkg -L pkg                # 패키지의 파일 목록
dpkg -S /etc/file          # 파일이 속한 패키지 검색
dpkg -s pkg                # 상태 · 정보
dpkg --info pkg.deb        # deb 파일 정보

# apt / apt-get — 고수준
apt update                 # 패키지 목록 갱신 (저장소 정보) ★
apt upgrade                # 설치된 패키지 업그레이드
apt full-upgrade           # 의존성 변경 포함 업그레이드
apt install nginx          # 설치
apt remove nginx           # 삭제
apt purge nginx            # 설정까지 삭제
apt autoremove             # 불필요한 의존 패키지 제거
apt search nginx           # 검색
apt show nginx             # 정보
apt list --installed       # 설치 목록
apt-cache search nginx     # 검색(구 방식)
apt-cache depends nginx    # 의존성
```

- 저장소 설정: **`/etc/apt/sources.list`**, `/etc/apt/sources.list.d/`
- ⚠️ 혼동 주의: `yum update`는 **패키지 업그레이드**지만, `apt update`는 **목록 갱신**입니다. 실제 업그레이드는 `apt upgrade`.

---

13. 소스 컴파일 설치 (Tarball)

표준 3단계 (순서 암기 ★★★)

```
tar zxvf pkg-1.0.tar.gz       # 1) 압축 해제
cd pkg-1.0
./configure --prefix=/usr/local/pkg   # 2) 환경 설정 (Makefile 생성)
make                                  # 3) 컴파일
make install                          # 4) 설치
```

단계	역할
`./configure`	시스템 환경 검사 → **Makefile 생성**. `--prefix`로 설치 경로 지정
`make`	Makefile에 따라 **소스 컴파일**
`make install`	컴파일된 결과를 지정 경로에 **복사·설치**
`make clean`	컴파일 중간 파일 삭제
`make uninstall`	설치 제거 (지원하는 경우)

출제 포인트

- **순서:** `configure` → `make` → `make install`
- `./configure`가 만드는 파일은 **Makefile**
- 설치 경로 지정 옵션은 `--prefix`
- 컴파일에는 gcc, make, 개발 헤더 패키지가 필요 → `yum groupinstall "Development Tools"`

관련 도구

도구	역할
`gcc`	C 컴파일러
`g++`	C++ 컴파일러
`make`	빌드 자동화
`cmake`	크로스 플랫폼 빌드 설정
`ldd`	실행 파일의 공유 라이브러리 의존성 확인
`ldconfig`	공유 라이브러리 캐시 갱신 (`/etc/ld.so.conf`)
`ar`	정적 라이브러리(.a) 생성

---

14. 마무리 체크 문제

1. vi, vim, emacs의 개발자? → **빌 조이 / 브람 무레나르 / 리처드 스톨만**
2. 커서 아래에 새 행을 만들며 입력 모드로? → `o`
3. 현재 행을 삭제하는 vi 명령? → `dd`
4. 3개 행을 복사하는 명령? → `3yy`
5. 문서 전체에서 문자열을 모두 치환? → `:%s/old/new/g`
6. 저장하지 않고 강제 종료? → `:q!`
7. vi에서 행 번호를 표시하는 설정? → `:set nu`
8. vim 설정을 영구 저장하는 파일? → `~/.vimrc`
9. rpm으로 설치된 전체 패키지 목록? → `rpm -qa`
10. 특정 파일이 어느 패키지에 속하는지 확인? → `rpm -qf 파일경로`
11. rpm에서 패키지에 포함된 파일 목록? → `rpm -ql 패키지명`
12. rpm 삭제 옵션? → `-e`
13. rpm이 의존성을 자동 해결하는가? → **아니오** (yum/dnf가 해결)
14. yum 저장소 설정 파일 위치? → `/etc/yum.repos.d/*.repo`
15. apt에서 저장소 목록만 갱신하는 명령? → `apt update`
16. 소스 컴파일 설치 3단계 순서? → `./configure` → `make` → `make install`
17. `./configure`가 생성하는 파일? → **Makefile**
18. 설치 경로를 지정하는 configure 옵션? → `--prefix`

07. X 윈도와 시스템 관리

1. X 윈도 시스템 개요

X 윈도(X Window System) = 유닉스/리눅스의 **네트워크 기반 GUI 시스템**

항목	내용
개발	1984년 MIT (Athena 프로젝트)
현재 버전	X11 (X11R7 등)
라이선스	**MIT 라이선스**
구현체	**XFree86** → **X.org (Xorg)** 로 주류 이동
최신 대안	**Wayland** (X의 후속, Xorg 대체 진행 중)
특징	**네트워크 투명성**, 클라이언트-서버 구조, 플랫폼 독립

X의 클라이언트-서버 구조 (헷갈림 주의 ★)

> **X 서버 = 디스플레이(모니터·키보드·마우스)가 있는 쪽 X 클라이언트 = 응용 프로그램(GUI 앱)**

즉 **내 앞의 모니터가 X 서버**이고, 원격 서버에서 돌아가는 프로그램이 X 클라이언트입니다. 일반적인 서버/클라이언트 개념과 반대라서 시험에 자주 나옵니다.

통신 프로토콜

X 프로토콜(XDMCP 포함) — TCP **6000번 포트**부터 사용 (`DISPLAY :0` → 6000, `:1` → 6001)

DISPLAY 환경변수

```
DISPLAY=호스트명:디스플레이번호.스크린번호
예: DISPLAY=:0.0          (로컬 첫 번째 디스플레이)
    DISPLAY=192.168.1.10:0.0
```

```
echo $DISPLAY
export DISPLAY=192.168.1.10:0.0     # 원격 디스플레이로 출력 지정
```

접근 제어

```
xhost +                     # 모든 호스트 허용 (보안상 위험)
xhost -                     # 모든 호스트 차단
xhost +192.168.1.10         # 특정 호스트 허용 ★
xhost -192.168.1.10         # 특정 호스트 차단
xauth list                  # 인증 쿠키(MIT-MAGIC-COOKIE) 목록
```

원격 X 사용

```
ssh -X user@host            # SSH X11 포워딩 ★ (안전한 방법)
ssh -Y user@host            # 신뢰된 포워딩
```

2. 윈도 매니저와 데스크톱 환경

윈도 매니저(Window Manager)

창의 테두리·이동·크기 조절 등 **창 관리만** 담당 예: **twm, mwm, fvwm, Enlightenment, Metacity, Mutter, Kwin, Compiz, i3, openbox**

데스크톱 환경(Desktop Environment)

윈도 매니저 + 파일 관리자 + 패널 + 응용 프로그램 **통합 환경**

환경	기반 툴킷	특징
GNOME	**GTK+**	심플·직관적, RHEL/Ubuntu 기본
KDE (Plasma)	**Qt**	화려하고 설정 옵션 많음
XFCE	GTK+	경량, 저사양 PC
LXDE / LXQt	GTK+ / Qt	초경량
Unity	GTK+	과거 Ubuntu 기본
MATE	GTK+	GNOME 2 포크
Cinnamon	GTK+	Linux Mint 기본

출제 포인트

- **GNOME = GTK+**, **KDE = Qt** (툴킷 매칭 단골)
- 윈도 매니저 ≠ 데스크톱 환경

---

3. 디스플레이 매니저와 X 실행

디스플레이 매니저 (그래픽 로그인 화면)

매니저	관련 환경
XDM	X 기본
GDM	GNOME
KDM / SDDM	KDE
LightDM	경량, Ubuntu

X 실행·설정 명령

```
startx                    # X 윈도 수동 시작
xinit                     # 저수준 X 시작
Xorg -configure            # xorg.conf 생성
X -configure
xrandr                    # 해상도 · 모니터 설정 확인/변경
xrandr -s 1920x1080
xdpyinfo                  # X 서버 정보
xwininfo                  # 창 정보
xkill                     # 클릭으로 창 강제 종료
xlsfonts                  # 폰트 목록
xset q                    # X 설정 조회
```

설정 파일

파일	역할
`/etc/X11/xorg.conf`	X 서버 설정 (최신 배포판은 자동 감지로 없는 경우가 많음)
`/etc/X11/xorg.conf.d/`	분할 설정
`~/.xinitrc`	사용자별 X 시작 스크립트
`~/.Xresources`	X 애플리케이션 리소스
`~/.xsession`	세션 시작 스크립트

xorg.conf 주요 섹션

`ServerLayout`, `Files`, `Module`, `InputDevice`, `Monitor`, `Device` (그래픽카드), `Screen`

X 관련 종료·전환

키	기능
`Ctrl + Alt + Backspace`	X 서버 강제 종료(설정에 따라 비활성)
`Ctrl + Alt + F1~F6`	텍스트 가상 콘솔로 전환
`Ctrl + Alt + F7` (또는 F2)	X 윈도로 복귀

GUI ↔ CLI 전환 (systemd)

```
systemctl isolate multi-user.target      # CLI로 전환
systemctl isolate graphical.target       # GUI로 전환
systemctl set-default multi-user.target  # 부팅 기본값을 CLI로
```

---

4. 사용자 계정 관리

관련 파일 (필드 구조 암기 ★★★)

`/etc/passwd` — 7개 필드

```
root:x:0:0:root:/root:/bin/bash
 (1) (2)(3)(4) (5)  (6)    (7)
```

필드	내용
1	사용자명(로그인 ID)
2	패스워드 → **`x` 는 shadow 파일 사용 의미**
3	**UID**
4	**GID**
5	설명(comment, 실명)
6	**홈 디렉터리**
7	**로그인 쉘**

`/etc/shadow` — 9개 필드 (암호화된 비밀번호)

```
root:$6$xyz...:19000:0:99999:7:::
 (1)   (2)     (3) (4) (5) (6)(7)(8)(9)
```

필드	내용
1	사용자명
2	**암호화된 패스워드** (`!` / `*` = 잠김, 빈칸 = 패스워드 없음)
3	마지막 변경일 (1970-01-01부터 일수)
4	**최소 사용일수** (변경 후 이 기간 내 재변경 불가)
5	**최대 사용일수** (만료일)
6	경고 일수 (만료 전 경고)
7	유예(비활성) 기간
8	계정 만료일
9	예약 필드

`/etc/group` — 4개 필드

```
wheel:x:10:user1,user2
 (1) (2)(3)   (4)
```

필드	내용
1	그룹명
2	그룹 패스워드(`x` = gshadow 사용)
3	**GID**
4	**그룹 멤버 목록**(2차 그룹)

기타

- `/etc/gshadow` — 그룹 패스워드
- `/etc/login.defs` — 계정 생성 기본 정책(UID 범위, 패스워드 만료 등)
- `/etc/default/useradd` — useradd 기본값
- **`/etc/skel/` — 새 계정 홈 디렉터리에 복사되는 기본 파일 템플릿 ★**

UID 관례 (출제 포인트)

UID	용도
0	**root (슈퍼유저)**
1 ~ 199 (또는 1~499)	시스템 계정
200 ~ 999	시스템 데몬 계정
1000 이상	**일반 사용자** (최신 배포판)

> UID를 0으로 만들면 root와 동일한 권한을 가집니다 (보안 점검 항목).

계정 관리 명령

```
useradd user1                       # 계정 생성
useradd -u 1500 user1               # UID 지정
useradd -g users user1              # 기본 그룹 지정
useradd -G wheel,dev user1          # 2차 그룹 지정
useradd -d /data/user1 user1        # 홈 디렉터리 지정
useradd -s /bin/bash user1          # 쉘 지정
useradd -s /sbin/nologin user1      # 로그인 불가 계정 ★
useradd -m user1                    # 홈 디렉터리 생성
useradd -M user1                    # 홈 디렉터리 생성 안 함
useradd -e 2026-12-31 user1         # 계정 만료일
useradd -c "Hong Gildong" user1     # 설명

passwd user1                        # 패스워드 설정
passwd -l user1                     # 계정 잠금(lock) ★
passwd -u user1                     # 잠금 해제(unlock)
passwd -d user1                     # 패스워드 삭제
passwd -S user1                     # 패스워드 상태 확인
passwd -e user1                     # 다음 로그인 시 강제 변경
passwd -n 7 -x 90 -w 7 user1        # 최소/최대/경고 일수

usermod -l newname oldname          # 이름 변경
usermod -u 2000 user1               # UID 변경
usermod -g users user1              # 기본 그룹 변경
usermod -aG wheel user1             # ★ 2차 그룹 추가 (-a 없으면 기존 그룹 제거!)
usermod -L user1                    # 계정 잠금
usermod -U user1                    # 잠금 해제
usermod -s /bin/csh user1           # 쉘 변경
usermod -d /new/home -m user1       # 홈 디렉터리 변경 + 이동
usermod -e 2026-12-31 user1         # 만료일

userdel user1                       # 계정 삭제 (홈 디렉터리 유지)
userdel -r user1                    # ★ 홈 디렉터리 · 메일까지 삭제

groupadd dev                        # 그룹 생성
groupadd -g 2000 dev                # GID 지정
groupmod -n newdev dev              # 그룹명 변경
groupmod -g 3000 dev                # GID 변경
groupdel dev                        # 그룹 삭제
gpasswd -a user1 dev                # 그룹에 사용자 추가
gpasswd -d user1 dev                # 그룹에서 사용자 제거
gpasswd -A user1 dev                # 그룹 관리자 지정

chage -l user1                      # 패스워드 만료 정보 조회 ★
chage -M 90 user1                   # 최대 사용일수 90일
chage -m 7 user1                    # 최소 사용일수
chage -W 7 user1                    # 경고 일수
chage -E 2026-12-31 user1           # 계정 만료일
chage -d 0 user1                    # 다음 로그인 시 패스워드 변경 강제

id user1                            # UID/GID/그룹 확인
groups user1                        # 소속 그룹
su - user1                          # 사용자 전환 (- 는 환경까지 전환) ★
su -c "command" user1               # 명령만 실행
sudo command                        # 관리자 권한으로 실행
visudo                              # /etc/sudoers 안전하게 편집
newgrp dev                          # 기본 그룹 임시 변경
```

```
pwck / grpck                    # passwd/group 파일 무결성 검사
pwconv / pwunconv               # shadow 사용/해제
```

출제 포인트

- `usermod -G` 만 쓰면 **기존 2차 그룹이 모두 사라집니다.** 추가할 때는 반드시 `-aG`
- `userdel -r` = 홈 디렉터리까지 삭제
- `su` vs `su -`: 하이픈을 붙이면 **대상 사용자의 환경변수까지 완전히 전환**
- 로그인 차단 쉘: `/sbin/nologin`, `/bin/false`

---

5. 로그 관리

주요 로그 파일 (`/var/log/`) — 암기 ★

파일	내용
`/var/log/messages`	**시스템 전반의 일반 로그** (RHEL 계열)
`/var/log/syslog`	시스템 일반 로그 (Debian 계열)
`/var/log/secure`	**인증·보안 관련** (로그인 성공/실패, su, sudo) — RHEL
`/var/log/auth.log`	인증 로그 (Debian)
`/var/log/boot.log`	부팅 메시지
`/var/log/dmesg`	커널 링 버퍼 메시지 (`dmesg` 명령)
`/var/log/cron`	cron 실행 로그
`/var/log/maillog`	메일 로그
`/var/log/wtmp`	**로그인/로그아웃 기록** (바이너리) → `last` 명령
`/var/log/btmp`	**로그인 실패 기록** (바이너리) → `lastb` 명령
`/var/log/lastlog`	**사용자별 최근 로그인** → `lastlog` 명령
`/var/run/utmp`	**현재 로그인 중인 사용자** → `who`, `w`
`/var/log/xferlog`	FTP 전송 로그
`/var/log/httpd/`	아파치 access_log, error_log

바이너리 로그는 cat으로 볼 수 없습니다 (출제 포인트)

로그 파일	확인 명령
`wtmp`	`last`
`btmp`	`lastb`
`lastlog`	`lastlog`
`utmp`	`who`, `w`, `users`

로그 데몬

데몬	설정 파일
`syslogd` (구)	`/etc/syslog.conf`
`rsyslogd` (현재)	**`/etc/rsyslog.conf`**
`journald` (systemd)	`/etc/systemd/journald.conf`, 조회는 `journalctl`

rsyslog 설정 형식

```
facility.priority       action
*.info;mail.none        /var/log/messages
authpriv.*              /var/log/secure
mail.*                  -/var/log/maillog
cron.*                  /var/log/cron
*.emerg                 :omusrmsg:*
kern.*                  @192.168.1.10      # 원격 전송
```

Facility (로그 발생 장소)

`kern`, `user`, `mail`, `daemon`, `auth`, `authpriv`, `syslog`, `lpr`, `news`, `uucp`, `cron`, `local0~7`

Priority (심각도 — 낮은 순서부터 암기 ★)

순위	레벨	의미
0	**emerg** (panic)	시스템 사용 불가 — 가장 심각
1	**alert**	즉시 조치 필요
2	**crit**	치명적
3	**err** (error)	오류
4	**warning** (warn)	경고
5	**notice**	정상이지만 주의
6	**info**	정보
7	**debug**	디버깅 — 가장 상세

> 암기: **emerg - alert - crit - err - warning - notice - info - debug** (심각 → 가벼움) `*.info`로 지정하면 **info 이상 (더 심각한) 모든 레벨**이 기록됩니다. `.none`은 해당 facility 제외, `=`는 해당 레벨만, `!`는 제외.

로그 순환 — logrotate

설정: `/etc/logrotate.conf`, `/etc/logrotate.d/`

```
/var/log/httpd/*log {
    daily              # 주기 (daily/weekly/monthly)
    rotate 4           # 4개 보관
    compress           # 압축
    missingok          # 없어도 에러 없음
    notifempty         # 비어있으면 순환 안 함
    create 0644 root root
}
```

journalctl (systemd)

```
journalctl                  # 전체 로그
journalctl -f               # 실시간
journalctl -u sshd          # 특정 유닛
journalctl -b               # 현재 부팅 이후
journalctl -p err           # err 이상만
journalctl --since "1 hour ago"
journalctl --since today
journalctl -n 50            # 최근 50줄
journalctl --disk-usage     # 로그 용량
```

---

6. 백업

백업 도구

명령	특징
`tar`	가장 일반적. 아카이브 + 압축
`dd`	**블록 단위 복제** — 디스크/파티션 전체 이미지
`dump` / `restore`	ext 파일시스템 전용, **증분 백업 레벨(0~9)** 지원
`rsync`	**증분·차등 동기화**, 네트워크 전송
`cpio`	아카이브 (find와 조합)
`mt`	테이프 장치 제어
`amanda`, `bacula`	네트워크 통합 백업 솔루션

```
# tar 백업/복원
tar zcvf /backup/home_$(date +%F).tar.gz /home
tar zxvf /backup/home.tar.gz -C /

# dd — 디스크 이미지
dd if=/dev/sda of=/backup/sda.img bs=1M        # 백업
dd if=/backup/sda.img of=/dev/sda bs=1M        # 복원
dd if=/dev/zero of=/swapfile bs=1M count=1024  # 파일 생성
# if=input file, of=output file, bs=block size, count=블록 수

# dump / restore
dump -0uf /backup/full.dump /home     # 레벨 0 = 전체 백업
dump -1uf /backup/inc1.dump /home     # 레벨 1 = 증분
restore -rf /backup/full.dump

# rsync
rsync -av /home/ /backup/home/               # 로컬 동기화
rsync -avz -e ssh /home/ user@host:/backup/  # 원격(압축 · SSH)
rsync -av --delete /home/ /backup/home/      # 원본에 없는 파일 삭제
```

rsync 옵션	의미
`-a`	아카이브 (권한·시간·링크 보존)
`-v`	상세 출력
`-z`	전송 시 압축
`-e ssh`	SSH를 통해 전송
`--delete`	원본에 없는 대상 파일 삭제
`-n` / `--dry-run`	실제 수행 없이 테스트

백업 종류

종류	설명
전체(Full)	모든 데이터 백업. 복구 간단, 용량·시간 많음
증분(Incremental)	**직전 백업 이후** 변경분만. 백업 빠름, **복구 복잡**
차등(Differential)	**최초 전체 백업 이후** 변경분 전체. 증분보다 용량 큼, 복구 단순

---

7. RAID (네트워크관리사에서도 출제)

레벨	방식	최소 디스크	특징
RAID 0	스트라이핑(Striping)	2개	**성능↑, 내결함성 없음**
RAID 1	미러링(Mirroring)	2개	**안정성↑, 가용 용량 50%**
RAID 2	해밍 코드 ECC	3개	실사용 안 함
RAID 3	바이트 단위 + 전용 패리티	3개	
RAID 4	블록 단위 + 전용 패리티	3개	패리티 디스크 병목
RAID 5	블록 스트라이핑 + **분산 패리티**	**3개**	성능·안정성 균형. **디스크 1개 장애 허용**
RAID 6	이중 분산 패리티	**4개**	**디스크 2개 장애 허용**
RAID 0+1 / 1+0(10)	스트라이핑 + 미러링	4개	고성능 + 고안정성

리눅스 소프트웨어 RAID 명령: `mdadm`

```
mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb1 /dev/sdc1 /dev/sdd1
mdadm --detail /dev/md0
cat /proc/mdstat
```

---

8. 프린터·주변장치 관리

인쇄 시스템

시스템	특징
LPD / LPRng	구형 BSD 계열
CUPS (Common UNIX Printing System)	**현재 표준**. IPP 프로토콜, **631번 포트**, 웹 관리(`http://localhost:631`)

인쇄 명령 (System V / BSD 계열 구분 ★)

기능	BSD 계열	System V 계열
인쇄 요청	`lpr file`	`lp file`
대기열 확인	`lpq`	`lpstat`
작업 취소	`lprm`	`cancel`
프린터 제어	`lpc`	-

```
lpr -P printer1 file.txt      # 특정 프린터로 인쇄
lpr -#3 file.txt              # 3부 인쇄
lpq -P printer1               # 대기열
lprm 12                       # 12번 작업 삭제
lpstat -a                     # 프린터 상태
lpadmin                       # 프린터 추가 · 설정
```

설정 파일: `/etc/cups/cupsd.conf`, `/etc/cups/printers.conf`, 구 시스템은 `/etc/printcap`

사운드

시스템	설명
OSS (Open Sound System)	구형
ALSA (Advanced Linux Sound Architecture)	**현재 표준 커널 사운드**
PulseAudio	사운드 서버(믹싱)
SANE	**스캐너** 표준 (`scanimage` , `xsane`)

장치 정보 확인 명령

```
lspci               # PCI 장치 목록 (그래픽 · NIC) ★
lsusb               # USB 장치 목록
lscpu               # CPU 정보
lsblk               # 블록 장치
lsmod               # 로드된 커널 모듈 목록 ★
modprobe module     # 모듈 로드(의존성 자동)
modprobe -r module  # 모듈 제거
insmod module.ko    # 모듈 삽입(의존성 미해결)
rmmod module        # 모듈 제거
modinfo module      # 모듈 정보
dmesg               # 커널 메시지 (장치 인식 확인)
dmesg ¦ tail
hdparm /dev/sda     # 하드디스크 파라미터
smartctl -a /dev/sda # 디스크 건강 상태
```

---

9. 마무리 체크 문제

1. X 윈도에서 모니터가 있는 쪽은 서버인가 클라이언트인가? → **서버**
2. X 윈도의 기본 포트 번호? → **6000**
3. GNOME과 KDE의 기반 툴킷? → **GTK+ / Qt**
4. `DISPLAY=:0.0` 의 의미? → 로컬 호스트의 0번 디스플레이, 0번 스크린
5. 특정 호스트에 X 접근을 허용하는 명령? → `xhost +192.168.1.10`

6. `/etc/passwd`의 필드 개수와 7번째 필드? → **7개, 로그인 쉘**
7. `/etc/shadow`의 2번째 필드? → **암호화된 패스워드**
8. 새 계정 홈에 복사되는 템플릿 디렉터리? → **`/etc/skel/`**
9. root의 UID? → **0**
10. 기존 2차 그룹을 유지하며 그룹을 추가하는 옵션? → **`usermod -aG`**
11. 홈 디렉터리까지 삭제하며 계정 삭제? → `userdel -r`
12. 패스워드 만료 정보를 확인하는 명령? → `chage -l`
13. 로그인 실패 기록 파일과 확인 명령? → **`/var/log/btmp`**, **`lastb`**
14. 인증 관련 로그 파일(RHEL)? → **`/var/log/secure`**
15. syslog 심각도 중 가장 심각한 것과 가장 낮은 것? → **emerg / debug**
16. 로그 순환을 담당하는 도구? → **logrotate**
17. 디스크 전체를 블록 단위로 복제하는 명령? → **`dd`**
18. 최소 3개 디스크로 분산 패리티를 쓰는 RAID? → **RAID 5**
19. 현재 표준 인쇄 시스템과 포트? → **CUPS, 631**
20. 로드된 커널 모듈 목록을 보는 명령? → `lsmod`

08. 네트워크 설정과 인터넷 활용

> **2차 시험 2과목(32문항, 40%)의 핵심 단원입니다.** 이 단원을 버리면 **과락(40% 미만)** 위험이 큽니다. 반드시 챙기세요.
> 네트워크관리사 2급과 겹치는 부분은 ../00_공통/중복 개념 정리.md에 더 자세히 있습니다.

---

1. 네트워크 기초 요약 (공통 노트 참조)

- OSI 7계층 / TCP-IP 4계층 매핑
- IP 주소 클래스, 사설 IP, 서브넷마스크
- TCP vs UDP, 3-way handshake
- 포트 번호

→ 위 내용은 **[00_공통/중복 개념 정리.md]** 에 정리되어 있습니다. 이 파일에서는 **리눅스 고유 부분**에 집중합니다.

---

2. 네트워크 토폴로지와 LAN 기술

토폴로지

형태	특징
버스(Bus)	하나의 케이블 공유. 설치 쉬움, 케이블 장애 시 전체 마비
스타(Star)	중앙 장비(허브/스위치) 연결. **현재 가장 일반적**. 중앙 고장 시 전체 마비
링(Ring)	원형 연결. Token Ring, FDDI
메시(Mesh)	모든 노드 상호 연결. 안정성 최고, 비용 높음
트리(Tree)	계층적 스타 확장

LAN 접근 제어 방식

방식	설명
CSMA/CD	**이더넷(유선)**. 충돌 탐지 후 재전송
CSMA/CA	**무선랜(Wi-Fi)**. 충돌 회피
Token Passing	토큰을 가진 노드만 전송 (Token Ring)

네트워크 규모별 분류

LAN(근거리) → **MAN**(도시권) → WAN(광역)

- VLAN: 논리적 LAN 분할
- VPN: 공용망을 통한 사설망 구성 (터널링)

무선랜 규격

규격	주파수	최대 속도
802.11b	2.4GHz	11Mbps
802.11a	5GHz	54Mbps
802.11g	2.4GHz	54Mbps
802.11n (Wi-Fi 4)	2.4/5GHz	600Mbps
802.11ac (Wi-Fi 5)	5GHz	6.9Gbps
802.11ax (Wi-Fi 6)	2.4/5GHz	9.6Gbps

무선 보안: WEP(취약) → WPA → **WPA2(AES/CCMP)** → WPA3

---

3. 리눅스 네트워크 설정 파일 (RHEL 계열 기준)

`/etc/sysconfig/network-scripts/ifcfg-eth0`

```
DEVICE=eth0                # 장치명
NAME=eth0
TYPE=Ethernet
ONBOOT=yes                 # 부팅 시 활성화 ★
BOOTPROTO=static           # static ¦ dhcp ¦ none ★
IPADDR=192.168.1.100       # IP 주소
NETMASK=255.255.255.0      # 서브넷마스크
PREFIX=24                  # (NETMASK 대체)
GATEWAY=192.168.1.1        # 기본 게이트웨이
DNS1=8.8.8.8
DNS2=168.126.63.1
HWADDR=00:0C:29:XX:XX:XX   # MAC 주소
```

항목	의미
`ONBOOT=yes`	부팅 시 자동 활성화
`BOOTPROTO=static`	고정 IP
`BOOTPROTO=dhcp`	DHCP 자동 할당

그 밖의 파일

파일	역할
`/etc/sysconfig/network`	호스트명, 기본 게이트웨이(구 방식)
`/etc/hostname`	호스트명
`/etc/hosts`	호스트명 ↔ IP 수동 매핑. **DNS보다 먼저 참조**
`/etc/resolv.conf`	DNS 서버 지정 (`nameserver` , `search` , `domain`)
`/etc/nsswitch.conf`	이름 해석 순서 (`hosts: files dns`)
`/etc/services`	서비스명 ↔ 포트
`/etc/protocols`	프로토콜명 ↔ 번호
`/etc/network/interfaces`	**Debian/Ubuntu 계열** NIC 설정

파일	역할
`/etc/netplan/*.yaml`	최신 Ubuntu 네트워크 설정

`/etc/resolv.conf` 예

```
nameserver 8.8.8.8
nameserver 168.126.63.1
search example.com
domain example.com
```

> nameserver는 최대 3개까지 유효합니다.

서비스 재시작

```
systemctl restart network          # RHEL 7
systemctl restart NetworkManager   # RHEL 7/8+
nmcli connection reload
ifdown eth0 && ifup eth0
```

---

4. 네트워크 설정·확인 명령

```
# IP 확인 · 설정
ifconfig                                # 전체 인터페이스
ifconfig eth0                           # 특정 인터페이스
ifconfig eth0 192.168.1.100 netmask 255.255.255.0 up   # 임시 설정
ifconfig eth0 up / down                 # 활성/비활성
ifconfig eth0 mtu 1400                  # MTU 변경
ip addr show                            # ★ 신 방식
ip addr add 192.168.1.100/24 dev eth0
ip addr del 192.168.1.100/24 dev eth0
ip link set eth0 up / down
ip -s link                              # 통계 포함

# 라우팅
route -n                                # 라우팅 테이블 (숫자로) ★
route add default gw 192.168.1.1        # 기본 게이트웨이 추가
route add -net 10.0.0.0/8 gw 192.168.1.254
route del default gw 192.168.1.1
ip route show                           # ★ 신 방식
ip route add default via 192.168.1.1
netstat -rn                             # 라우팅 테이블

# 연결 진단
ping 8.8.8.8
ping -c 4 8.8.8.8                       # 4회만 (count) ★
ping -i 2 host                          # 2초 간격
ping -s 1000 host                       # 패킷 크기 지정
traceroute 8.8.8.8                      # 경로 추적 ★
tracepath 8.8.8.8
mtr 8.8.8.8                             # 실시간 경로 + 손실률

# 소켓 · 포트
netstat -an                             # 전체 연결
netstat -antp                           # ★ TCP + 숫자 + 프로세스
```

```
netstat -tulnp                          # ★ LISTEN 중인 포트 확인
netstat -i                              # 인터페이스 통계
netstat -s                              # 프로토콜별 통계
ss -antp                                # ★ netstat 후속
ss -tulnp
lsof -i :80                             # 80번 포트 사용 프로세스

# DNS 조회
nslookup www.google.com
dig www.google.com                      # 상세 조회 ★
dig @8.8.8.8 example.com MX             # 특정 서버 · 레코드 지정
dig -x 8.8.8.8                          # 역방향 조회
host www.google.com
hostname                                # 호스트명
hostnamectl set-hostname server1        # 호스트명 영구 변경 ★

# ARP
arp -a                                  # ARP 캐시
arp -d 192.168.1.1                      # 항목 삭제
ip neigh show                           # 신 방식

# NIC 상태
ethtool eth0                            # 속도 · 듀플렉스 · 링크 상태 ★
ethtool -s eth0 speed 100 duplex full
mii-tool eth0                           # 링크 상태(구)

# nmcli (NetworkManager CLI)
nmcli device status
nmcli connection show
nmcli connection up eth0
nmcli con mod eth0 ipv4.addresses 192.168.1.100/24
nmcli con mod eth0 ipv4.gateway 192.168.1.1
nmcli con mod eth0 ipv4.method manual

# 파일 전송 · 다운로드
wget http://example.com/file.tar.gz     # 다운로드 ★
wget -c URL                             # 이어받기
wget -r URL                             # 재귀적(사이트 미러)
curl -O URL                             # 다운로드
curl -I URL                             # 헤더만 확인
scp file user@host:/path                # SSH 기반 복사 ★
scp -r dir user@host:/path              # 디렉터리
scp user@host:/path/file .              # 가져오기
sftp user@host                          # 대화형 보안 FTP
rsync -avz src/ user@host:dst/          # 동기화

# 원격 접속
ssh user@host                           # ★
ssh -p 2222 user@host                   # 포트 지정
ssh -i key.pem user@host                # 키 파일 지정
ssh-keygen -t rsa -b 4096               # 키 쌍 생성 ★
ssh-copy-id user@host                   # 공개키 전송
telnet host 80                          # 포트 연결 테스트
nc -zv host 80                          # 포트 스캔 · 테스트

# 방화벽
firewall-cmd --state
firewall-cmd --list-all
firewall-cmd --permanent --add-port=80/tcp     # ★
firewall-cmd --permanent --add-service=http
```

```
firewall-cmd --reload
iptables -L -n                                  # 규칙 목록
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

출제 포인트 요약

목적	명령
LISTEN 중인 포트 확인	`netstat -tulnp` 또는 `ss -tulnp`
기본 게이트웨이 확인	`route -n` 또는 `ip route`
DNS 상세 조회	`dig`
NIC 속도·듀플렉스 확인	`ethtool`
호스트명 영구 변경	`hostnamectl set-hostname`
4회만 ping	`ping -c 4`

---

5. 주요 서버 서비스와 데몬

서비스	데몬	포트	설정 파일
웹 (Apache)	`httpd`	80/443	`/etc/httpd/conf/httpd.conf`
웹 (Nginx)	`nginx`	80/443	`/etc/nginx/nginx.conf`
SSH	`sshd`	**22**	`/etc/ssh/sshd_config` ★
FTP (vsftpd)	`vsftpd`	20/21	`/etc/vsftpd/vsftpd.conf`
메일 (송신)	`sendmail`, `postfix`	25	`/etc/mail/sendmail.cf`, `/etc/postfix/main.cf`
메일 (수신)	`dovecot`	110/143	`/etc/dovecot/dovecot.conf`
DNS (BIND)	`named`	**53**	`/etc/named.conf`
DHCP	`dhcpd`	67/68	`/etc/dhcp/dhcpd.conf`
NFS	`nfs`, `rpcbind`	2049	**`/etc/exports`** ★
Samba	`smbd`, `nmbd`	139/445	**`/etc/samba/smb.conf`** ★
DB (MySQL/MariaDB)	`mysqld`, `mariadb`	3306	`/etc/my.cnf`
PostgreSQL	`postgres`	5432	`postgresql.conf`
프록시 (Squid)	`squid`	3128	`/etc/squid/squid.conf`
NTP	`ntpd`, `chronyd`	123	`/etc/ntp.conf`, `/etc/chrony.conf`
인쇄 (CUPS)	`cupsd`	631	`/etc/cups/cupsd.conf`
슈퍼데몬	`xinetd`	-	`/etc/xinetd.conf`, `/etc/xinetd.d/`
로그	`rsyslogd`	514	`/etc/rsyslog.conf`
cron	`crond`	-	`/etc/crontab`

파일 공유 서비스 비교 (출제 단골)

구분	NFS	Samba (SMB/CIFS)
용도	**UNIX/Linux ↔ UNIX/Linux**	**Linux ↔ Windows**
설정 파일	`/etc/exports`	`/etc/samba/smb.conf`
데몬	`nfs`, `rpcbind` (portmap)	`smbd`, `nmbd`
포트	2049	137~139, 445
마운트	`mount -t nfs host:/share /mnt`	`mount -t cifs //host/share /mnt`

```
# NFS 서버
vi /etc/exports
/data  192.168.1.0/24(rw,sync,no_root_squash)
exportfs -ra              # 설정 반영
showmount -e host         # 공개 목록 확인

# Samba
smbclient -L //host       # 공유 목록
smbpasswd -a user1        # Samba 계정 추가
testparm                  # smb.conf 문법 검사
```

SSH 보안 설정 (`/etc/ssh/sshd_config`)

```
Port 22
PermitRootLogin no            # root 직접 로그인 차단 ★
PasswordAuthentication no     # 키 인증만 허용
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers user1 user2
```

---

6. 인터넷 서비스 프로토콜 정리

프로토콜	용도	포트
HTTP / HTTPS	웹	80 / 443
FTP	파일 전송 (능동/수동 모드)	21(제어) / 20(데이터)
SFTP / SCP	SSH 기반 보안 전송	22
TFTP	간이 파일 전송(UDP)	69
SMTP	메일 발신	25
POP3	메일 수신(다운로드 후 서버에서 삭제)	110
IMAP	메일 수신(서버에 보관, 다중 기기)	143
Telnet	원격 접속 (**평문 — 보안 취약**)	23
SSH	원격 접속 (**암호화**)	22
DNS	이름 해석	53
DHCP	IP 자동 할당	67/68
SNMP	네트워크 관리	161/162
NTP	시간 동기화	123

프로토콜	용도	포트
LDAP	디렉터리	389
RDP	윈도우 원격데스크톱	3389
VNC	원격 데스크톱	5900+

FTP 능동(Active) vs 수동(Passive) 모드

모드	데이터 연결 방향
Active	**서버 → 클라이언트** (서버 20번 포트에서 연결 시도). 클라이언트 방화벽에 막히기 쉬움
Passive	**클라이언트 → 서버** (서버가 임의 포트를 알려줌). 방화벽 환경에 유리

POP3 vs IMAP

구분	POP3	IMAP
메일 보관	클라이언트로 다운로드	**서버에 보관**
다중 기기	부적합	**적합**
포트	110	143

---

7. DHCP 동작 과정 (순서 암기 ★)

```
1. DHCP DISCOVER  — 클라이언트 → 브로드캐스트로 서버 탐색
2. DHCP OFFER     — 서버 → IP 제안
3. DHCP REQUEST   — 클라이언트 → 해당 IP 사용 요청
4. DHCP ACK       — 서버 → 최종 승인
```

암기: **DORA** (Discover - Offer - Request - Ack)

- DHCP 실패 시: **169.254.x.x (APIPA)** 자동 할당
- 임대 갱신: 임대 기간의 50% 시점에 갱신 시도(RENEW), 87.5%에 REBIND
- 리눅스 클라이언트: `dhclient`, `dhclient -r` (반납)

---

8. 리눅스 응용 분야 (2과목 후반부)

서버 분야

웹 서버(Apache, Nginx), DB(MySQL/MariaDB, PostgreSQL), 메일, DNS, 파일 서버(NFS/Samba), 프록시(Squid)

클라우드·가상화

기술	설명
KVM	리눅스 커널 내장 전가상화 하이퍼바이저
Xen	반가상화 중심 하이퍼바이저
VirtualBox / VMware	호스트형 가상화
Docker	컨테이너 기반 — 커널 공유, 경량
Kubernetes(K8s)	컨테이너 오케스트레이션
OpenStack	오픈소스 IaaS 클라우드 플랫폼

클라우드 서비스 모델 (출제 포인트)

모델	제공 범위	예
IaaS	인프라(서버·스토리지·네트워크)	AWS EC2, OpenStack
PaaS	개발 플랫폼·런타임	Heroku, Google App Engine
SaaS	완성된 소프트웨어	Gmail, Office 365

임베디드·IoT

라즈베리파이, 안드로이드, 셋톱박스, 공유기 펌웨어(OpenWrt), Tizen

클러스터링·빅데이터

기술	설명
베어울프(Beowulf) 클러스터	고계산용(HPC) 클러스터
LVS	부하분산 클러스터
HA 클러스터	고가용성, 페일오버
Hadoop / HDFS	분산 저장·처리
Spark	인메모리 분산 처리
MapReduce	분산 처리 프로그래밍 모델

임베디드 관련 용어

- **크로스 컴파일**: 개발 PC에서 다른 아키텍처(ARM 등)용 바이너리 생성
- **부트로더**: U-Boot 등
- **RTOS**: 실시간 운영체제

---

9. 보안 기초

용어	설명
방화벽(Firewall)	패킷 필터링. `iptables`, `nftables`, `firewalld`
IDS	침입 **탐지** 시스템 (탐지·경고만)
IPS	침입 **방지** 시스템 (차단까지)
SELinux	강제 접근 제어(MAC). 모드: enforcing / permissive / disabled (`getenforce`, `setenforce 0`)

용어	설명
TCP Wrapper	서비스 접근 제어. `/etc/hosts.allow`, `/etc/hosts.deny` ★
PAM	인증 모듈. `/etc/pam.d/`
DoS / DDoS	서비스 거부 공격
스니핑	패킷 도청 (`tcpdump`, `wireshark`)
스푸핑	주소 위장 (IP/ARP/DNS 스푸핑)
SetUID 점검	`find / -perm -4000`

TCP Wrapper 규칙 (출제 포인트)

```
# /etc/hosts.allow
sshd : 192.168.1.

# /etc/hosts.deny
ALL : ALL
```

> **`hosts.allow` 를 먼저 확인하고, 그다음 `hosts.deny` 를 확인합니다.** 둘 다 없으면 허용.

SELinux 명령

```
getenforce                  # 현재 모드
setenforce 0                # permissive로 임시 전환
setenforce 1                # enforcing
sestatus                    # 상태 상세
# 영구 변경: /etc/selinux/config 의 SELINUX= 값 수정
```

---

10. 마무리 체크 문제

1. 부팅 시 NIC를 자동 활성화하는 ifcfg 항목? → **`ONBOOT=yes`**
2. 고정 IP를 쓰려면 `BOOTPROTO` 값? → **`static`**
3. DNS 서버를 지정하는 파일? → **`/etc/resolv.conf`**
4. `/etc/hosts` 와 DNS 중 먼저 참조되는 것? → **`/etc/hosts`**
5. LISTEN 중인 포트를 확인하는 명령? → `netstat -tulnp` / `ss -tulnp`
6. 기본 게이트웨이를 확인하는 명령 2가지? → `route -n`, `ip route`
7. ping을 4회만 보내는 옵션? → `-c 4`
8. NIC 속도·듀플렉스를 확인하는 명령? → `ethtool`
9. 호스트명을 영구히 바꾸는 명령? → `hostnamectl set-hostname`
10. NFS의 설정 파일? → **`/etc/exports`**
11. Samba의 설정 파일과 용도? → **`/etc/samba/smb.conf`**, 윈도우와 파일 공유
12. SSH 서버 설정 파일과 root 로그인 차단 항목? → `/etc/ssh/sshd_config`, `PermitRootLogin no`
13. DHCP 4단계 순서? → **Discover → Offer → Request → Ack (DORA)**
14. FTP 수동 모드의 데이터 연결 방향? → **클라이언트 → 서버**
15. 메일을 서버에 보관해 여러 기기에서 보기 적합한 프로토콜? → **IMAP (143)**
16. 유선 이더넷과 무선랜의 접근 제어 방식? → **CSMA/CD / CSMA/CA**
17. TCP Wrapper의 두 설정 파일과 확인 순서? → `hosts.allow` → `hosts.deny`

18. SELinux 모드 3가지? → enforcing / permissive / disabled
19. IaaS·PaaS·SaaS 중 인프라만 제공하는 모델? → **IaaS**
20. 컨테이너와 가상머신의 핵심 차이? → **컨테이너는 호스트 커널을 공유**하므로 가볍고 빠름

09. 명령어 치트시트 (인쇄용)

> **인쇄해서 들고 다니세요.** 2차 1과목 48문항 중 대부분이 여기서 나옵니다. 명령어 자체는 쉽습니다. **옵션 문자를 정확히 구분**하는 것이 승부처입니다.

A. 헷갈리기 쉬운 옵션 — 최우선 암기 ★★★

명령	옵션	의미	함정
`tar`	`z` / `j` / `J`	gzip / bzip2 / xz	소문자 j = bzip2, **대문자 J = xz**
`tar`	`c` / `x` / `t` / `r` / `u`	생성 / 풀기 / **목록** / 추가 / 갱신	`t` = list
`ps`	`-ef` / `aux`	System V / **BSD** 스타일	`aux` 는 하이픈 없음
`grep`	`-v`	**일치하지 않는** 행	`-i` 는 대소문자 무시
`grep`	`-c` / `-l`	개수 / 파일명만	
`find`	`-atime` / `-mtime` / `-ctime`	접근 / 내용수정 / 속성변경	
`find`	`-perm -4000`	SetUID 검색	
`kill`	기본값	**15 (SIGTERM)**	-9 아님
`cp`	`-r` / `-p` / `-a`	재귀 / 속성보존 / 아카이브	디렉터리는 `-r` 필수
`rm`	`-r` / `-f` / `-i`	재귀 / 강제 / 확인	
`wc`	`-l` / `-w` / `-c`	행 / 단어 / 바이트	`-l` =line
`df` / `du`	-	**파일시스템** 여유 / **디렉터리** 사용량	혼동 최다
`usermod`	`-aG` vs `-G`	추가 vs **덮어쓰기**	`-a` 없으면 기존 그룹 소실
`userdel`	`-r`	홈 디렉터리까지 삭제	
`rpm`	`-qf` / `-ql`	파일 → 패키지 / 패키지 → 파일	
`rpm`	`-i` / `-U` / `-F` / `-e`	설치/업그레이드/갱신만/삭제	
`apt`	`update` vs `upgrade`	목록 갱신 vs 업그레이드	yum과 반대
`head` / `tail`	`-n` / `-f`	줄 수 / **실시간 추적**	`tail -f` 단골
`ln`	`-s`	심볼릭 링크	없으면 하드 링크
`sort`	`-n` / `-r` / `-u` / `-k`	숫자 / 역순 / 중복제거 / 필드	
`chmod`	`4/2/1`	SetUID / SetGID / Sticky	
`nice`	`-20 ~ 19`	**작을수록 우선순위 높음**	음수는 root만

B. 파일·디렉터리

```
pwd                         현재 경로
cd ~ / cd .. / cd -         홈 / 상위 / 직전
ls -al                      전체 상세 목록
ls -i                       inode 번호
ls -lrt                     시간 역순 상세
mkdir -p a/b/c              중간 경로까지 생성
rmdir dir                   빈 디렉터리만 삭제
rm -rf dir                  재귀 강제 삭제
cp -r src dst               디렉터리 복사
cp -p file dst              속성 보존 복사
cp -a src dst               아카이브 복사(-dpR)
mv a b                      이동/이름변경
touch file                  빈 파일 생성 / 시간 갱신
file test.txt               파일 종류 판별
stat file                   상세 속성(3가지 시간)
ln origin hard              하드 링크
ln -s origin soft           심볼릭 링크
```

C. 내용 보기·편집

```
cat -n file                 행 번호 포함 출력
tac file                    역순 출력
more / less file            페이지 단위 (less는 뒤로도 가능)
head -5 file                앞 5줄
tail -20 file               뒤 20줄
tail -f /var/log/messages   실시간 로그 추적 ★
wc -l file                  행 수
nl file                     행 번호 붙이기
od -c file                  덤프
strings binary              문자열 추출
```

D. 검색

```
find /home -name "*.txt"            이름
find / -iname "abc*"                대소문자 무시
find / -type f / -type d / -type l  파일/디렉터리/링크
find / -user u1 / -group g1         소유자/그룹
find / -perm 644 / -perm -4000      권한 / SetUID
find / -size +10M / -size -100k     크기
find / -mtime -7                    7일 이내 수정
find / -empty                       빈 파일
find /tmp -name "*.tmp" -exec rm {} \;   찾아 실행
find . -maxdepth 2                  깊이 제한

grep -i / -v / -n / -c / -l / -r / -w    무시/반대/번호/개수/파일명/재귀/단어
grep "^root" file                   시작
grep "bash$" file                   끝
egrep "a|b" file                    확장 정규식

which ls        PATH 내 경로
whereis ls      바이너리+소스+man
locate passwd   DB 기반 빠른 검색
updatedb        locate DB 갱신
type cd         명령어 종류
```

E. 압축·아카이브

```
gzip file / gunzip file.gz          .gz
bzip2 file / bunzip2 file.bz2       .bz2
xz file / unxz file.xz              .xz
zip -r a.zip dir / unzip a.zip      .zip (묶기 가능)
zcat file.gz                        압축 상태로 보기

tar cvf a.tar dir       묶기
tar xvf a.tar           풀기
tar tvf a.tar           목록 ★
tar zcvf a.tar.gz dir   gzip 묶기 ★
tar zxvf a.tar.gz       gzip 풀기 ★
tar jcvf a.tar.bz2 dir  bzip2 묶기
tar jxvf a.tar.bz2      bzip2 풀기
tar Jcvf a.tar.xz dir   xz 묶기
tar xvf a.tar -C /opt   지정 위치에 풀기
```

F. 권한·소유권

```
chmod 755 file / chmod u+x file
chmod -R 755 dir
chmod 4755 file      SetUID
chmod 2755 dir       SetGID
chmod 1777 dir       Sticky Bit
chown u1:g1 file
chown -R u1 dir
chgrp g1 file
umask / umask 022
chattr +i file       삭제 · 수정 불가
lsattr file
```

8진수 ↔ 기호 즉답표

```
7 = rwx    6 = rw-    5 = r-x    4 = r--
3 = -wx    2 = -w-    1 = --x    0 = ---
```

umask 계산

```
파일     = 666 − umask
디렉터리 = 777 − umask
022 → 644 / 755      027 → 640 / 750      077 → 600 / 700
```

G. 프로세스

```
ps -ef / ps aux                전체 프로세스
ps -ef ¦ grep httpd
pstree -p                      계층 트리
pgrep / pidof httpd            PID 검색
top                            실시간 (k=kill, r=renice, M=메모리, P=CPU, q=종료)
kill PID / kill -9 PID         종료 / 강제 종료
kill -l                        시그널 목록
killall httpd / pkill -f pat
jobs -l                        작업 목록
fg %1 / bg %1                  포그라운드/백그라운드
command &                      백그라운드 실행
nohup command &                로그아웃 후에도 유지
nice -n 10 cmd                 우선순위 지정 실행
renice 5 -p 1234               실행 중 우선순위 변경
free -h / uptime / vmstat 1
lsof -i :80                    포트 사용 프로세스
```

시그널 번호

```
1 HUP(재읽기)  2 INT(Ctrl+C)  3 QUIT  9 KILL(강제)
15 TERM(기본)  18 CONT  19 STOP  20 TSTP(Ctrl+Z)
```

H. 작업 예약

```
crontab -e / -l / -r            편집/목록/삭제
분 시 일 월 요일 명령
0 3 * * *          매일 03:00
30 2 1 * *         매월 1일 02:30
*/10 * * * *       10분마다
0 9 * * 1          매주 월요일 09:00
0 22 * * 1-5       평일 22:00

at 10:00 / atq / atrm 3         1회성 예약
```

`/etc/crontab` 은 **6개 필드**(5개 시간 + 실행 사용자 + 명령)

I. 디스크·파일시스템

```
df -h / df -i / df -T           여유공간 / inode / 타입
du -sh dir / du -h --max-depth=1
fdisk -l / fdisk /dev/sdb       파티션 (n,p,w,d,t,q)
parted / gdisk                  GPT
mkfs.ext4 /dev/sdb1             파일시스템 생성
mkfs -t xfs /dev/sdb1
mkswap /dev/sdb2 / swapon -s    스왑
mount /dev/sdb1 /mnt
mount -o remount,rw /
mount -a / umount /mnt
fsck -y /dev/sdb1               검사 · 복구 (언마운트 후!)
tune2fs -l /dev/sdb1
blkid / lsblk                   UUID / 블록 장치 트리
quotacheck -mcug /home / quotaon
edquota -u u1 / repquota /home
```

/etc/fstab 필드

```
장치 마운트포인트 파일시스템 옵션 dump fsck순서
/dev/sda2  /home  ext4  defaults  1  2
```

J. 계정 관리

```
useradd -m -s /bin/bash -G wheel u1
useradd -s /sbin/nologin svc      로그인 불가 계정
passwd u1 / passwd -l u1 / -u u1  설정/잠금/해제
usermod -aG dev u1                2차 그룹 추가 ★
usermod -l new old / -s /bin/csh
userdel -r u1                     홈까지 삭제
groupadd -g 2000 dev / groupdel dev
gpasswd -a u1 dev / -d u1 dev
chage -l u1 / -M 90 u1 / -E 날짜
id / groups / who / w / last / lastb / lastlog
su - u1 / sudo cmd / visudo
```

계정 파일 필드

```
/etc/passwd (7): 이름:x:UID:GID:설명:홈:쉘
/etc/shadow (9): 이름:암호:최종변경:최소:최대:경고:유예:만료:예약
/etc/group  (4): 그룹명:x:GID:멤버목록
/etc/skel/     : 새 계정 홈 템플릿
```

K. 패키지

```
# RPM
rpm -ivh pkg.rpm / -Uvh / -e pkg
rpm -qa ¦ grep httpd              설치 확인
rpm -qi / -ql / -qc / -qd httpd   정보/파일/설정/문서
rpm -qf /path/file                파일 → 패키지 ★
rpm -qR httpd                     의존성
rpm -V httpd                      검증

# YUM / DNF
yum -y install httpd / remove / update
yum list installed / info / search
yum provides */httpd.conf
yum groupinstall "Development Tools"
yum clean all / repolist / history
설정: /etc/yum.repos.d/*.repo

# DPKG / APT
dpkg -i pkg.deb / -r / -P / -l / -L / -S
apt update / upgrade / install / remove / purge / autoremove
설정: /etc/apt/sources.list

# 소스 컴파일
tar zxvf pkg.tar.gz
./configure --prefix=/usr/local   → Makefile 생성
make → make install → make clean
```

L. 네트워크

```
ifconfig / ip addr show
ifconfig eth0 192.168.1.10 netmask 255.255.255.0 up
ip addr add 192.168.1.10/24 dev eth0
route -n / ip route show
route add default gw 192.168.1.1
ping -c 4 host
traceroute host / tracepath / mtr
netstat -antp / -tulnp / -rn
ss -antp / -tulnp
nslookup / dig / dig -x IP / host
arp -a / ip neigh
hostnamectl set-hostname server1
ethtool eth0
nmcli device status / con show
wget -c URL / curl -O URL / curl -I URL
scp file u@host:/path / scp -r
ssh u@host / ssh -p 2222 / ssh-keygen -t rsa
firewall-cmd --permanent --add-port=80/tcp; firewall-cmd --reload
iptables -L -n
```

네트워크 설정 파일

```
/etc/sysconfig/network-scripts/ifcfg-eth0   (RHEL)
  DEVICE ONBOOT=yes BOOTPROTO=static IPADDR NETMASK GATEWAY DNS1
/etc/hosts        호스트명 매핑 (DNS보다 우선)
/etc/resolv.conf  nameserver 지정
/etc/services     서비스↔포트
/etc/nsswitch.conf 이름해석 순서
```

M. 서비스·로그

```
systemctl start/stop/restart/reload/status httpd
systemctl enable/disable httpd
systemctl is-active / is-enabled / list-unit-files
systemctl get-default / set-default multi-user.target
systemctl isolate graphical.target
journalctl -u sshd / -f / -b / -p err / -n 50

로그: /var/log/messages(일반) secure(인증) boot.log cron maillog
바이너리: wtmp→last, btmp→lastb, lastlog→lastlog, utmp→who
rsyslog: /etc/rsyslog.conf  형식: facility.priority  action
심각도: emerg alert crit err warning notice info debug
logrotate: /etc/logrotate.conf
```

N. vi 에디터

```
입력   i I a A o O s S R cw
이동   h j k l  w b e  0 ^ $  gg G nG  H M L  Ctrl+f/b
삭제   x X  dd ndd  dw  D(d$) dG
복사   yy nyy yw   붙여넣기 p P
취소   u  U  Ctrl+r   반복 .
검색   /pat ?pat n N
치환   :s/a/b/   :s/a/b/g   :%s/a/b/g ★   :1,10s/a/b/g
저장   :w :wq :wq! ZZ :q :q! :x
설정   :set nu / nonu / ai / ts=4 / ic / hls
파일   :r file  :r !date  :!cmd  :sh
영구설정 ~/.vimrc
```

O. 시스템 종료·부팅

```
shutdown -h now / -h +10 / -r now / -c
halt / poweroff / reboot / init 0 / init 6
systemctl poweroff / reboot
sync

런레벨: 0종료 1단일 2NFS없음 3CLI 4미사용 5GUI 6재부팅
타깃: poweroff rescue multi-user graphical reboot
runlevel / who -r / init 3
```

P. 기타

```
date +%Y-%m-%d / cal / uptime
uname -a / uname -r             커널 버전
lscpu / lspci / lsusb / lsblk / lsmod
dmesg ¦ tail                    커널 메시지
modprobe mod / rmmod / modinfo
alias ll='ls -l' / unalias / \rm
history / !! / !52 / !$ / Ctrl+R
export VAR=값 / env / set / unset
source ~/.bashrc  (= . ~/.bashrc)
man 5 passwd / info / --help
echo $? $$ $# $1 $SHELL $PATH
tee out.txt / tee -a out.txt
cmd > f / >> f / 2> f / > f 2>&1 / 2>/dev/null
```

Q. 텍스트 처리

```
sort -n -r -u -k3 -t:
uniq -c / -d / -u
cut -d: -f1 /etc/passwd
cut -c1-5 file
tr 'a-z' 'A-Z' < file / tr -d '\r' / tr -s ' '
paste / join / split -l 100 / diff / cmp / comm
sed 's/a/b/g' / sed -i / sed -n '5,10p' / sed '3d'
awk '{print $1}' / awk -F: '{print $1,$3}'
awk 'NR==5' / awk 'END{print NR}' / awk '$3>500'
```

R. 백업

```
tar zcvf /backup/home.tar.gz /home
dd if=/dev/sda of=/backup/sda.img bs=1M
dump -0uf /backup/f.dump /home   (레벨 0 = 전체)
restore -rf /backup/f.dump
rsync -avz -e ssh /home/ u@host:/backup/
rsync -av --delete src/ dst/
```

S. 최종 D-Day 암기 리스트 (시험 전 10분)

```
umask 022 → 파일 644, 디렉터리 755
SetUID=4 SetGID=2 Sticky=1
/usr/bin/passwd = 4755 (SetUID)
/tmp = 1777 (Sticky)
kill 기본 = 15(TERM), 강제 = 9(KILL)
Ctrl+C=2  Ctrl+Z=20
nice: -20(최우선) ~ 19, 음수는 root만
런레벨 3=CLI, 5=GUI
passwd 7필드 / shadow 9필드 / group 4필드
fstab: 장치 마운트점 종류 옵션 dump fsck
crontab: 분 시 일 월 요일
/etc/crontab만 6필드(사용자 추가)
tar: z=gzip j=bzip2 J=xz t=목록
rpm -qf(파일→패키지) -ql(패키지→파일)
configure → make → make install
X서버 = 모니터 있는 쪽, 포트 6000
GNOME=GTK+, KDE=Qt
wtmp→last, btmp→lastb
로그 심각도: emerg alert crit err warning notice info debug
DHCP: DORA
NFS=/etc/exports, Samba=/etc/samba/smb.conf
RAID5 최소 3개(1개 장애), RAID6 최소 4개(2개 장애)
```

10. 빈출 암기노트 — 시험 전날용

> 시험 전날 이 파일만 1회독하세요. 자주 출제되는 함정과 숫자만 모았습니다.

---

1. 인물 매칭 (거의 매 회차 출제)

인물	업적
리누스 토발즈	리눅스 커널 (1991)
리처드 스톨만	GNU 프로젝트(1983), FSF, GPL, **emacs**
켄 톰슨	UNIX
데니스 리치	C 언어, UNIX 재작성
앤드루 타넨바움	MINIX
빌 조이	**csh**, **vi**
브람 무레나르	**vim**
브라이언 폭스	**bash**
데이비드 콘	ksh
스티븐 본	sh (Bourne Shell)
마크 어윙	GNOME 창시
마티아스 에트리히	KDE 창시
이안 머독	Debian
마크 셔틀워스	Ubuntu (Canonical)
패트릭 볼커딩	Slackware

> 함정 최다: **vi = 빌 조이 / vim = 브람 무레나르 / emacs = 리처드 스톨만**

---

2. 숫자 암기 (전부 외우세요)

항목	값
리눅스 커널 최초 발표	**1991년**
GNU 프로젝트 시작	1983년
root의 UID / GID	**0 / 0**
일반 사용자 UID 시작	**1000** (구 배포판 500)
X 윈도 기본 포트	**6000**
CUPS 포트	**631**
nice 값 범위	**-20 ~ 19** (기본 0)
kill 기본 시그널	**15 (SIGTERM)**
MTU 기본값	**1500**

항목	값
IPv4 / IPv6 주소 길이	32 / **128** 비트
MAC 주소 길이	48비트
기본 umask (root / 일반)	022 / 002
`/etc/passwd` 필드 수	**7**
`/etc/shadow` 필드 수	**9**
`/etc/group` 필드 수	**4**
`/etc/fstab` 필드 수	**6**
사용자 crontab 필드 수	**5**
`/etc/crontab` 필드 수	**6**
MBR 최대 디스크 용량	**2TB**
MBR 주 파티션 최대 개수	**4**
논리 파티션 시작 번호	**5**
RAID 5 최소 디스크	**3**
RAID 6 최소 디스크	**4**
1차 시험 문항/시간	50문항 / 60분
2차 시험 문항/시간	**80문항 / 100분**
합격 기준	60점, 2차는 과목당 40% 이상

---

3. 최다 함정 10선

① `df` vs `du`

- `df` = 파일시스템(파티션) 단위 **여유 공간**
- `du` = 디렉터리/파일 단위 **사용량**

② `apt update` vs `yum update`

- `apt update` = **패키지 목록만 갱신** (업그레이드는 `apt upgrade`)
- `yum update` = **실제 업그레이드**

③ `usermod -G` vs `-aG`

- `-G` 만 쓰면 **기존 2차 그룹이 전부 사라짐**
- 추가할 때는 반드시 `-aG`

④ X 윈도의 서버/클라이언트

- **X 서버 = 모니터·키보드가 있는 내 쪽**
- X 클라이언트 = 응용 프로그램
- 일반 상식과 **반대**

⑤ `tar` 압축 문자

- `z` = gzip, `j` = bzip2, `J` **(대문자)** = xz

- `t` = **목록 보기** (테스트가 아님)

⑥ ext2 vs ext3

- 차이는 **저널링(Journaling) 유무**. ext2에는 없음.

⑦ 하드 링크의 제약

- **디렉터리에 만들 수 없고, 다른 파일시스템에도 만들 수 없음**
- 원본 삭제해도 데이터 유지 (심볼릭 링크는 깨짐)

⑧ 대문자 `S` / `T`

- `rwS` = SetUID는 있으나 **실행 권한(x) 없음** → 작동 안 함
- 소문자 `s` / `t` 가 정상

⑨ `rpm` 삭제 시 인자

- 설치는 **파일명**(`httpd-2.4.rpm`), 삭제는 **패키지명**(`httpd`)

⑩ 사설 IP 172 대역

- **172.16 ~ 172.31**까지만. `172.32.x.x` 는 공인 IP

---

4. 파일 경로 암기

용도	경로
사용 가능한 쉘 목록	`/etc/shells`
사용자 계정 정보	`/etc/passwd`
암호화된 패스워드	`/etc/shadow`
그룹 정보	`/etc/group`
계정 생성 기본 정책	`/etc/login.defs`
새 계정 홈 템플릿	**`/etc/skel/`**
sudo 권한 설정	`/etc/sudoers` (`visudo` 로 편집)
마운트 정보(부팅 시)	`/etc/fstab`
현재 마운트 상태	`/etc/mtab` , `/proc/mounts`
호스트명 ↔ IP	`/etc/hosts`
DNS 서버 지정	`/etc/resolv.conf`
서비스명 ↔ 포트	`/etc/services`
이름 해석 순서	`/etc/nsswitch.conf`
NIC 설정 (RHEL)	`/etc/sysconfig/network-scripts/ifcfg-eth0`
NIC 설정 (Debian)	`/etc/network/interfaces`
전역 쉘 환경	`/etc/profile` , `/etc/bashrc`
개인 환경변수	`~/.bash_profile`
개인 별칭·함수	`~/.bashrc`

용도	경로
로그아웃 시 실행	`~/.bash_logout`
명령 이력	`~/.bash_history`
vim 설정	`~/.vimrc`
반복 예약 작업	`/etc/crontab`, `/var/spool/cron/`
cron 허용·거부	`/etc/cron.allow`, `/etc/cron.deny`
로그 설정	`/etc/rsyslog.conf`
로그 순환	`/etc/logrotate.conf`
yum 저장소	`/etc/yum.repos.d/*.repo`
apt 저장소	`/etc/apt/sources.list`
SSH 서버 설정	`/etc/ssh/sshd_config`
NFS 공유 설정	**`/etc/exports`**
Samba 설정	**`/etc/samba/smb.conf`**
Apache 설정	`/etc/httpd/conf/httpd.conf`
DNS(BIND) 설정	`/etc/named.conf`
DHCP 설정	`/etc/dhcp/dhcpd.conf`
TCP Wrapper	`/etc/hosts.allow`, `/etc/hosts.deny`
SELinux 설정	`/etc/selinux/config`
X 서버 설정	`/etc/X11/xorg.conf`
GRUB2 설정	`/boot/grub2/grub.cfg`, `/etc/default/grub`
커널 이미지	`/boot/vmlinuz-*`
CPU 정보	`/proc/cpuinfo`
메모리 정보	`/proc/meminfo`
커널 버전	`/proc/version`
스왑 정보	`/proc/swaps`
RAID 상태	`/proc/mdstat`

---

5. 시그널 · 런레벨 · 로그 레벨

시그널

```
1  HUP   설정 재읽기 / 터미널 종료
2  INT   Ctrl+C
3  QUIT  Ctrl+\ (코어 덤프)
9  KILL  강제 종료 (무시 불가)
15 TERM  정상 종료 (kill 기본값)
18 CONT  계속 실행
19 STOP  중지 (무시 불가)
20 TSTP  Ctrl+Z
```

런레벨 ↔ systemd 타깃

```
0 halt        → poweroff.target
1 single      → rescue.target
2 NFS없는 다중 → multi-user.target
3 CLI         → multi-user.target
4 미사용
5 GUI         → graphical.target
6 reboot      → reboot.target
```

로그 심각도 (심각 → 경미)

```
emerg → alert → crit → err → warning → notice → info → debug
```

---

6. 퍼미션 즉답 훈련 (5초 내 답하기)

문제	답
`rwxr-xr-x`	755
`rw-r--r--`	644
`rwx------`	700
`rw-rw-r--`	664
`r-xr-x---`	550
640	`rw-r-----`
711	`rwx--x--x`
600	`rw-------`
umask 022 → 파일	644
umask 022 → 디렉터리	755
umask 027 → 파일	640
umask 027 → 디렉터리	750
umask 077 → 파일	600
umask 002 → 디렉터리	775
SetUID + 755	4755 → `rwsr-xr-x`
Sticky + 777	1777 → `rwxrwxrwt`
SetGID + 755	2755 → `rwxr-sr-x`

---

7. crontab 즉답 훈련

표현	의미
`0 0 * * *`	매일 자정
`0 3 * * *`	매일 03:00
`30 4 * * 0`	매주 일요일 04:30

표현	의미
`0 9 * * 1-5`	평일 09:00
`*/15 * * * *`	15분마다
`0 */6 * * *`	6시간마다
`15 14 1 * *`	매월 1일 14:15
`0 0 1 1 *`	매년 1월 1일 자정
`0 12 * * 6`	매주 토요일 12:00
매월 15일 22:30	`30 22 15 * *`
5분마다	`*/5 * * * *`

요일: **0 또는 7 = 일요일**, 1 = 월요일 … 6 = 토요일

---

8. 2차 시험 2과목(리눅스 활용) 과락 방지 핵심 20개

1. X 서버 = 모니터가 있는 쪽, 포트 6000, `DISPLAY=:0.0`
2. GNOME = GTK+, KDE = Qt
3. 디스플레이 매니저: XDM, GDM, KDM/SDDM, LightDM
4. `xhost +IP` = 특정 호스트 X 접근 허용
5. 윈도 매니저 ≠ 데스크톱 환경
6. `ONBOOT=yes`, `BOOTPROTO=static|dhcp`
7. `/etc/resolv.conf` = DNS 지정, `/etc/hosts`가 DNS보다 우선
8. `netstat -tulnp` / `ss -tulnp` = LISTEN 포트
9. `route -n` / `ip route` = 게이트웨이 확인
10. `ping -c 4`, `traceroute`, `dig`, `ethtool`
11. NFS = `/etc/exports` (Unix↔Unix), Samba = `smb.conf` (Linux↔Windows)
12. SSH 22, Telnet 23(평문), SFTP/SCP 22
13. DHCP = **DORA**, 실패 시 169.254.x.x
14. FTP: Active(서버→클라), Passive(클라→서버)
15. POP3(110) = 다운로드, IMAP(143) = 서버 보관
16. CSMA/CD = 유선, CSMA/CA = 무선
17. TCP Wrapper: `hosts.allow` → `hosts.deny` 순서
18. SELinux: enforcing / permissive / disabled
19. IaaS / PaaS / SaaS 구분
20. 컨테이너는 **호스트 커널 공유** → 가볍고 빠름

---

9. 시험장에서의 전략

1. **쉬운 명령어 문제부터 먼저 푸세요.** 80문항 100분 = 문항당 75초. 헤매면 안 됩니다.
2. **모르는 문제는 표시하고 넘어가세요.** 끝까지 한 바퀴 돌고 돌아옵니다.
3. **옵션 문자를 소리 내어 읽듯 확인하세요.** `-j`와 `-J`, `-e`와 `-E`를 눈으로만 보면 틀립니다.
4. **2과목(네트워크·활용)에서 최소 13문항 이상 맞히는 것**을 목표로 잡으세요. 과락선이 12.8점입니다.

5. 확실한 오답을 먼저 지우는 소거법이 잘 통하는 시험입니다.

---

10. 셀프 모의 테스트 (답 가리고 풀기)

#	문제	답
1	리눅스 커널 창시자와 연도	리누스 토발즈, 1991
2	vim 개발자	브람 무레나르
3	2차 저작물 공개를 강제하는 라이선스	GPL
4	커널 2.4.18은?	안정 버전
5	`.deb` 사용 계열	Debian
6	MBR 최대 용량	2TB
7	RHEL7+ 기본 파일시스템	XFS
8	런레벨 5	GUI 다중 사용자
9	`tar jxvf` 의 압축 방식	bzip2
10	목록만 보는 tar 옵션	`t`
11	SetUID 검색 명령	`find / -perm -4000`
12	umask 027 → 디렉터리 권한	750
13	`/tmp` 의 특수 권한	Sticky Bit (1777)
14	inode에 없는 정보	파일 이름
15	root도 못 지우게 하는 명령	`chattr +i`
16	fstab 6번째 필드	fsck 순서
17	리눅스 기본 쉘	bash
18	`$?` 의 의미	직전 명령 종료 상태
19	kill 기본 시그널	15 (TERM)
20	Ctrl+Z 시그널 번호	20 (TSTP)
21	nice 최우선 값	-20
22	`30 4 * * 0`	매주 일요일 04:30
23	`/etc/crontab` 필드 수	6
24	1회성 예약 목록 확인	`atq`
25	vi 전체 치환	`:%s/old/new/g`
26	저장 없이 강제 종료	`:q!`
27	파일 → 소속 패키지 확인	`rpm -qf`
28	configure가 만드는 파일	Makefile
29	X 윈도 포트	6000
30	KDE의 툴킷	Qt
31	`/etc/skel/` 의 역할	새 계정 홈 템플릿
32	2차 그룹 추가 옵션	`usermod -aG`

#	문제	답
33	로그인 실패 기록 확인	`lastb` (btmp)
34	가장 심각한 로그 레벨	emerg
35	디스크 블록 단위 복제	`dd`
36	RAID 5 최소 디스크 수	3
37	CUPS 포트	631
38	DHCP 4단계	DORA
39	Windows와 파일 공유 서비스	Samba
40	IMAP 포트	143